

```
In [12]: x.dtype
```

```
Out[12]: dtype('int64')
```

Why is `x` comprised of integers? This is because we created `x` by passing in exclusively integers to the `np.array()` function. If we had passed in any decimals, then we would have obtained an array of *floating point numbers* (i.e. real-valued numbers).

floating
point

```
In [13]: np.array([[1, 2], [3.0, 4]]).dtype
```

```
Out[13]: dtype('float64')
```

Typing `fun?` will cause `Python` to display documentation associated with the function `fun`, if it exists. We can try this for `np.array()`.

```
In [14]: np.array?
```

This documentation indicates that we could create a floating point array by passing a `dtype` argument into `np.array()`.

dtype

```
In [15]: np.array([[1, 2], [3, 4]], float).dtype
```

```
Out[15]: dtype('float64')
```

The array `x` is two-dimensional. We can find out the number of rows and columns by looking at its `shape` attribute.

shape

```
In [16]: x.shape
```

```
Out[16]: (2, 2)
```

A *method* is a function that is associated with an object. For instance, given an array `x`, the expression `x.sum()` sums all of its elements, using the `sum()` method for arrays. The call `x.sum()` automatically provides `x` as the first argument to its `sum()` method.

method

.sum()

```
In [17]: x = np.array([1, 2, 3, 4])
         x.sum()
```

```
Out[17]: 10
```

We could also sum the elements of `x` by passing in `x` as an argument to the `np.sum()` function.

np.sum()

```
In [18]: x = np.array([1, 2, 3, 4])
         np.sum(x)
```

```
Out[18]: 10
```

As another example, the `reshape()` method returns a new array with the same elements as `x`, but a different shape. We do this by passing in a `tuple`

.reshape()
tuple

in our call to `reshape()`, in this case `(2, 3)`. This tuple specifies that we would like to create a two-dimensional array with 2 rows and 3 columns.²

In what follows, the `\n` character creates a *new line*.

```
In [19]: x = np.array([1, 2, 3, 4, 5, 6])
print('beginning x:\n', x)
x_reshape = x.reshape((2, 3))
print('reshaped x:\n', x_reshape)
```

```
beginning x:
[1 2 3 4 5 6]
reshaped x:
[[1 2 3]
 [4 5 6]]
```

The previous output reveals that `numpy` arrays are specified as a sequence of *rows*. This is called *row-major ordering*, as opposed to *column-major ordering*.

`Python` (and hence `numpy`) uses 0-based indexing. This means that to access the top left element of `x_reshape`, we type in `x_reshape[0,0]`.

```
In [20]: x_reshape[0, 0]
```

```
Out[20]: 1
```

Similarly, `x_reshape[1,2]` yields the element in the second row and the third column of `x_reshape`.

```
In [21]: x_reshape[1, 2]
```

```
Out[21]: 6
```

Similarly, `x[2]` yields the third entry of `x`.

Now, let's modify the top left element of `x_reshape`. To our surprise, we discover that the first element of `x` has been modified as well!

```
In [22]: print('x before we modify x_reshape:\n', x)
print('x_reshape before we modify x_reshape:\n', x_reshape)
x_reshape[0, 0] = 5
print('x_reshape after we modify its top left element:\n',
      x_reshape)
print('x after we modify top left element of x_reshape:\n', x)
```

```
Out[22]: x before we modify x_reshape:
[1 2 3 4 5 6]
x_reshape before we modify x_reshape:
[[1 2 3]
 [4 5 6]]
x_reshape after we modify its top left element:
[[5 2 3]
```

²Like lists, tuples represent a sequence of objects. Why do we need more than one way to create a sequence? There are a few differences between tuples and lists, but perhaps the most important is that elements of a tuple cannot be modified, whereas elements of a list can be.

```
[4 5 6]]
x after we modify top left element of x_reshape:
[5 2 3 4 5 6]
```

Modifying `x_reshape` also modified `x` because the two objects occupy the same space in memory.

We just saw that we can modify an element of an array. Can we also modify a tuple? It turns out that we cannot — and trying to do so introduces an *exception*, or error.

```
In [23]: my_tuple = (3, 4, 5)
         my_tuple[0] = 2
```

exception

```
TypeError: 'tuple' object does not support item assignment
```

We now briefly mention some attributes of arrays that will come in handy. An array's `shape` attribute contains its dimension; this is always a tuple. The `ndim` attribute yields the number of dimensions, and `T` provides its transpose.

```
In [24]: x_reshape.shape, x_reshape.ndim, x_reshape.T
```

```
Out[24]: ((2, 3),
          2,
          array([[5, 4],
                 [2, 5],
                 [3, 6]]))
```

Notice that the three individual outputs `(2,3)`, `2`, and `array([[5, 4], [2, 5], [3,6]])` are themselves output as a tuple.

We will often want to apply functions to arrays. For instance, we can compute the square root of the entries using the `np.sqrt()` function:

```
In [25]: np.sqrt(x)
```

np.sqrt()

```
Out[25]: array([2.24, 1.41, 1.73, 2., 2.24, 2.45])
```

We can also square the elements:

```
In [26]: x**2
```

```
Out[26]: array([25, 4, 9, 16, 25, 36])
```

We can compute the square roots using the same notation, raising to the power of $1/2$ instead of 2.

```
In [27]: x**0.5
```

```
Out[27]: array([2.24, 1.41, 1.73, 2., 2.24, 2.45])
```

Throughout this book, we will often want to generate random data. The `np.random.normal()` function generates a vector of random normal variables. We can learn more about this function by looking at the help page, via a call to `np.random.normal?`. The first line of the help page reads `normal(loc=0.0, scale=1.0, size=None)`. This *signature* line tells us that the function's ar-

np.random.
normal()

signature

guments are `loc`, `scale`, and `size`. These are *keyword* arguments, which means that when they are passed into the function, they can be referred to by name (in any order).³ By default, this function will generate random normal variable(s) with mean (`loc`) 0 and standard deviation (`scale`) 1; furthermore, a single random variable will be generated unless the argument to `size` is changed.

We now generate 50 independent random variables from a $N(0,1)$ distribution.

```
In [28]: x = np.random.normal(size=50)
x
```

```
Out[28]: array([-1.19,  0.41,  0.9 , -0.44, -0.9 , -0.38,  0.13,  1.87,
        -0.35,  1.16,  0.79, -0.97, -1.21,  0.06, -1.62, -0.6 ,
        -0.77, -2.12,  0.38, -1.22, -0.06, -1.97, -1.74, -0.56,
         1.7 , -0.95,  0.56,  0.35,  0.87,  0.88, -1.66, -0.32,
        -0.3 , -1.36,  0.92, -0.31,  1.28, -1.94,  1.07,  0.07,
         0.79, -0.46,  2.19, -0.27, -0.64,  0.85,  0.13,  0.46,
        -0.09,  0.7 ])
```

We create an array `y` by adding an independent $N(50,1)$ random variable to each element of `x`.

```
In [29]: y = x + np.random.normal(loc=50, scale=1, size=50)
```

The `np.corrcoef()` function computes the correlation matrix between `x` and `y`. The off-diagonal elements give the correlation between `x` and `y`. `np.corrcoef()`

```
In [30]: np.corrcoef(x, y)
```

```
Out[30]: array([[1.   , 0.69],
        [0.69, 1.   ]])
```

If you're following along in your own *Jupyter* notebook, then you probably noticed that you got a different set of results when you ran the past few commands. In particular, each time we call `np.random.normal()`, we will get a different answer, as shown in the following example.

```
In [31]: print(np.random.normal(scale=5, size=2))
print(np.random.normal(scale=5, size=2))
```

```
Out[31]: [4.28 2.59]
[4.62 -2.54]
```

In order to ensure that our code provides exactly the same results each time it is run, we can set a *random seed* using the `np.random.default_rng()` function. This function takes an arbitrary, user-specified integer argument. If we set a random seed before generating random data, then re-running our code will yield the same results. The object `rng` has essentially all the random seed
`np.random.default_rng()`

³Python also uses *positional* arguments. Positional arguments do not need to use a keyword. To see an example, type in `np.sum?`. We see that `a` is a positional argument, i.e. this function assumes that the first unnamed argument that it receives is the array to be summed. By contrast, `axis` and `dtype` are keyword arguments: the position in which these arguments are entered into `np.sum()` does not matter.

random number generating methods found in `np.random`. Hence, to generate normal data we use `rng.normal()`.

```
In [32]: rng = np.random.default_rng(1303)
         print(rng.normal(scale=5, size=2))
         rng2 = np.random.default_rng(1303)
         print(rng2.normal(scale=5, size=2))
```

```
Out [32]: [4.09 -1.07 ]
         [4.09 -1.07 ]
```

Throughout the labs in this book, we use `np.random.default_rng()` whenever we perform calculations involving random quantities within `numpy`. In principle, this should enable the reader to exactly reproduce the stated results. However, as new versions of `numpy` become available, it is possible that some small discrepancies may occur between the output in the labs and the output from `numpy`.

The `np.mean()`, `np.var()`, and `np.std()` functions can be used to compute the mean, variance, and standard deviation of arrays. These functions are also available as methods on the arrays.

`np.mean()`
`np.var()`
`np.std()`

```
In [33]: rng = np.random.default_rng(3)
         y = rng.standard_normal(10)
         np.mean(y), y.mean()
```

```
Out [33]: (-0.11, -0.11)
```

```
In [34]: np.var(y), y.var(), np.mean((y - y.mean())**2)
```

```
Out [34]: (2.72, 2.72, 2.72)
```

Notice that by default `np.var()` divides by the sample size n rather than $n - 1$; see the `ddof` argument in `np.var?`.

```
In [35]: np.sqrt(np.var(y)), np.std(y)
```

```
Out [35]: (1.65, 1.65)
```

The `np.mean()`, `np.var()`, and `np.std()` functions can also be applied to the rows and columns of a matrix. To see this, we construct a 10×3 matrix of $N(0, 1)$ random variables, and consider computing its row sums.

```
In [36]: X = rng.standard_normal((10, 3))
         X
```

```
Out [36]: array([[ 0.23, -0.35, -0.28],
                 [-0.67, -1.06, -0.39],
                 [ 0.48, -0.24,  0.96],
                 [-0.2 ,  0.02,  1.55],
                 [ 0.55, -0.51, -0.18],
                 [ 0.54,  1.94, -0.27],
                 [-0.24,  1. , -0.89],
                 [-0.29,  0.88,  0.58],
                 [ 0.09,  0.67, -2.83],
                 [ 1.02, -0.96, -1.67]])
```

Since arrays are row-major ordered, the first axis, i.e. `axis=0`, refers to its rows. We pass this argument into the `mean()` method for the object `X`.

```
In [37]: X.mean(axis=0) .mean()
```

```
Out[37]: array([0.15, 0.14, -0.34])
```

The following yields the same result.

```
In [38]: X.mean(0)
```

```
Out[38]: array([0.15, 0.14, -0.34])
```

2.3.4 Graphics

In `Python`, common practice is to use the library `matplotlib` for graphics. However, since `Python` was not written with data analysis in mind, the notion of plotting is not intrinsic to the language. We will use the `subplots()` function from `matplotlib.pyplot` to create a figure and the axes onto which we plot our data. For many more examples of how to make plots in `Python`, readers are encouraged to visit matplotlib.org/stable/gallery/.

`matplotlib`

In `matplotlib`, a plot consists of a *figure* and one or more *axes*. You can think of the figure as the blank canvas upon which one or more plots will be displayed: it is the entire plotting window. The *axes* contain important information about each plot, such as its *x*- and *y*-axis labels, title, and more. (Note that in `matplotlib`, the word *axes* is not the plural of *axis*: a plot's *axes* contains much more information than just the *x*-axis and the *y*-axis.)

figure
axes

We begin by importing the `subplots()` function from `matplotlib`. We use this function throughout when creating figures. The function returns a tuple of length two: a figure object as well as the relevant axes object. We will typically pass `figsize` as a keyword argument. Having created our axes, we attempt our first plot using its `plot()` method. To learn more about it, type `ax.plot?`.

`subplots()`

`.plot()`

```
In [39]: from matplotlib.pyplot import subplots
fig, ax = subplots(figsize=(8, 8))
x = rng.standard_normal(100)
y = rng.standard_normal(100)
ax.plot(x, y);
```

We pause here to note that we have *unpacked* the tuple of length two returned by `subplots()` into the two distinct variables `fig` and `ax`. Unpacking is typically preferred to the following equivalent but slightly more verbose code:

```
In [40]: output = subplots(figsize=(8, 8))
fig = output[0]
ax = output[1]
```

We see that our earlier cell produced a line plot, which is the default. To create a scatterplot, we provide an additional argument to `ax.plot()`, indicating that circles should be displayed.

```
In [41]: fig, ax = subplots(figsize=(8, 8))
         ax.plot(x, y, 'o');
```

Different values of this additional argument can be used to produce different colored lines as well as different linestyles.

As an alternative, we could use the `ax.scatter()` function to create a scatterplot. `.scatter()`

```
In [42]: fig, ax = subplots(figsize=(8, 8))
         ax.scatter(x, y, marker='o');
```

Notice that in the code blocks above, we have ended the last line with a semicolon. This prevents `ax.plot(x, y)` from printing text to the notebook. However, it does not prevent a plot from being produced. If we omit the trailing semi-colon, then we obtain the following output:

```
In [43]: fig, ax = subplots(figsize=(8, 8))
         ax.scatter(x, y, marker='o')
```

```
Out[43]: <matplotlib.collections.PathCollection at 0x7fb3d9c8f310>
         Figure(432x288)
```

In what follows, we will use trailing semicolons whenever the text that would be output is not germane to the discussion at hand.

To label our plot, we make use of the `set_xlabel()`, `set_ylabel()`, and `set_title()` methods of `ax`. `.set_xlabel()`

```
In [44]: fig, ax = subplots(figsize=(8, 8))
         ax.scatter(x, y, marker='o')
         ax.set_xlabel("this is the x-axis")
         ax.set_ylabel("this is the y-axis")
         ax.set_title("Plot of X vs Y");
         .set_ylabel()
         .set_title()
```

Having access to the figure object `fig` itself means that we can go in and change some aspects and then redisplay it. Here, we change the size from (8, 8) to (12, 3).

```
fig.set_size_inches(12,3)
fig
```

Occasionally we will want to create several plots within a figure. This can be achieved by passing additional arguments to `subplots()`. Below, we create a 2×3 grid of plots in a figure of size determined by the `figsize` argument. In such situations, there is often a relationship between the axes in the plots. For example, all plots may have a common x -axis. The `subplots()` function can automatically handle this situation when passed the keyword argument `sharex=True`. The `axes` object below is an array pointing to different plots in the figure.

```
In [45]: fig, axes = subplots(nrows=2,
                             ncols=3,
                             figsize=(15, 5))
```

We now produce a scatter plot with 'o' in the second column of the first row and a scatter plot with '+' in the third column of the second row.

```
In [46]: axes[0,1].plot(x, y, 'o')
axes[1,2].scatter(x, y, marker='+')
fig
```

Type `subplots?` to learn more about `subplots()`.

To save the output of `fig`, we call its `savefig()` method. The argument `dpi` is the dots per inch, used to determine how large the figure will be in pixels. `.savefig()`

```
In [47]: fig.savefig("Figure.png", dpi=400)
fig.savefig("Figure.pdf", dpi=200);
```

We can continue to modify `fig` using step-by-step updates; for example, we can modify the range of the x -axis, re-save the figure, and even re-display it.

```
In [48]: axes[0,1].set_xlim([-1,1])
fig.savefig("Figure_updated.jpg")
fig
```

We now create some more sophisticated plots. The `ax.contour()` method produces a *contour plot* in order to represent three-dimensional data, similar to a topographical map. It takes three arguments: `.contour()`
contour plot

- A vector of `x` values (the first dimension),
- A vector of `y` values (the second dimension), and
- A matrix whose elements correspond to the `z` value (the third dimension) for each pair of `(x,y)` coordinates.

To create `x` and `y`, we'll use the command `np.linspace(a, b, n)`, which returns a vector of `n` numbers starting at `a` and ending at `b`. `np.linspace()`

```
In [49]: fig, ax = subplots(figsize=(8, 8))
x = np.linspace(-np.pi, np.pi, 50)
y = x
f = np.multiply.outer(np.cos(y), 1 / (1 + x**2))
ax.contour(x, y, f);
```

We can increase the resolution by adding more levels to the image.

```
In [50]: fig, ax = subplots(figsize=(8, 8))
ax.contour(x, y, f, levels=45);
```

To fine-tune the output of the `ax.contour()` function, take a look at the help file by typing `?plt.contour`.

The `ax.imshow()` method is similar to `ax.contour()`, except that it produces a color-coded plot whose colors depend on the `z` value. This is known as a *heatmap*, and is sometimes used to plot temperature in weather forecasts. `.imshow()`
heatmap

```
In [51]: fig, ax = subplots(figsize=(8, 8))
ax.imshow(f);
```


2.3.5 Sequences and Slice Notation

As seen above, the function `np.linspace()` can be used to create a sequence of numbers.

```
In [52]: seq1 = np.linspace(0, 10, 11)
         seq1
```

```
Out [52]: array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
```

The function `np.arange()` returns a sequence of numbers spaced out by `step`. If `step` is not specified, then a default value of 1 is used. Let's create a sequence that starts at 0 and ends at 10. `np.arange()`

```
In [53]: seq2 = np.arange(0, 10)
         seq2
```

```
Out [53]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Why isn't 10 output above? This has to do with *slice* notation in `Python`. Slice notation is used to index sequences such as lists, tuples and arrays. Suppose we want to retrieve the fourth through sixth (inclusive) entries of a string. We obtain a slice of the string using the indexing notation `[3:6]`. slice

```
In [54]: "hello world"[3:6]
```

```
Out [54]: 'lo '
```

In the code block above, the notation `3:6` is shorthand for `slice(3,6)` when used inside `[]`.

```
In [55]: "hello world"[slice(3,6)]
```

```
Out [55]: 'lo '
```

You might have expected `slice(3,6)` to output the fourth through seventh characters in the text string (recalling that `Python` begins its indexing at zero), but instead it output the fourth through sixth. This also explains why the earlier `np.arange(0, 10)` command output only the integers from 0 to 9. See the documentation `slice?` for useful options in creating slices.

2.3.6 Indexing Data

To begin, we create a two-dimensional `numpy` array.

```
In [56]: A = np.array(np.arange(16)).reshape((4, 4))
         A
```

```
Out [56]: array([[ 0,  1,  2,  3],
                 [ 4,  5,  6,  7],
                 [ 8,  9, 10, 11],
                 [12, 13, 14, 15]])
```

Typing `A[1,2]` retrieves the element corresponding to the second row and third column. (As usual, `Python` indexes from 0.)

```
In [57]: A[1,2]
```

```
Out[57]: 6
```

The first number after the open-bracket symbol `[` refers to the row, and the second number refers to the column.

Indexing Rows, Columns, and Submatrices

To select multiple rows at a time, we can pass in a list specifying our selection. For instance, `[1,3]` will retrieve the second and fourth rows:

```
In [58]: A[[1,3]]
```

```
Out[58]: array([[ 4,  5,  6,  7],
               [12, 13, 14, 15]])
```

To select the first and third columns, we pass in `[0,2]` as the second argument in the square brackets. In this case we need to supply the first argument : which selects all rows.

```
In [59]: A[:, [0,2]]
```

```
Out[59]: array([[ 0,  2],
               [ 4,  6],
               [ 8, 10],
               [12, 14]])
```

Now, suppose that we want to select the submatrix made up of the second and fourth rows as well as the first and third columns. This is where indexing gets slightly tricky. It is natural to try to use lists to retrieve the rows and columns:

```
In [60]: A[[1,3], [0,2]]
```

```
Out[60]: array([ 4, 14])
```

Oops — what happened? We got a one-dimensional array of length two identical to

```
In [61]: np.array([A[1,0], A[3,2]])
```

```
Out[61]: array([ 4, 14])
```

Similarly, the following code fails to extract the submatrix comprised of the second and fourth rows and the first, third, and fourth columns:

```
In [62]: A[[1,3], [0,2,3]]
```

```
IndexError: shape mismatch: indexing arrays could not be broadcast
together with shapes (2,) (3,)
```

We can see what has gone wrong here. When supplied with two indexing lists, the `numpy` interpretation is that these provide pairs of i, j indices for a series of entries. That is why the pair of lists must have the same length. However, that was not our intent, since we are looking for a submatrix.

One easy way to do this is as follows. We first create a submatrix by subsetting the rows of `A`, and then on the fly we make a further submatrix by subsetting its columns.

```
In [63]: A[[1,3]][:, [0,2]]
```

```
Out[63]: array([[ 4,  6],
               [12, 14]])
```

There are more efficient ways of achieving the same result.

The *convenience function* `np.ix_()` allows us to extract a submatrix using lists, by creating an intermediate *mesh* object.

```
In [64]: idx = np.ix_([1,3], [0,2,3])
         A[idx]
```

convenience
function
`np.ix_()`
mesh

```
Out[64]: array([[ 4,  6,  7],
               [12, 14, 15]])
```

Alternatively, we can subset matrices efficiently using slices. The slice `1:4:2` captures the second and fourth items of a sequence, while the slice `0:3:2` captures the first and third items (the third element in a slice sequence is the step size).

```
In [65]: A[1:4:2, 0:3:2]
```

```
Out[65]: array([[ 4,  6],
               [12, 14]])
```

Why are we able to retrieve a submatrix directly using slices but not using lists? Its because they are different `Python` types, and are treated differently by `numpy`. Slices can be used to extract objects from arbitrary sequences, such as strings, lists, and tuples, while the use of lists for indexing is more limited.

Boolean Indexing

In `numpy`, a *Boolean* is a type that equals either `True` or `False` (also represented as 1 and 0, respectively). The next line creates a vector of 0's, represented as Booleans, of length equal to the first dimension of `A`.

Boolean

```
In [66]: keep_rows = np.zeros(A.shape[0], bool)
         keep_rows
```

```
Out[66]: array([False,  False,  False,  False])
```

We now set two of the elements to `True`.

```
In [67]: keep_rows[[1,3]] = True
         keep_rows
```

```
Out[67]: array([False,  True, False,  True])
```

Note that the elements of `keep_rows`, when viewed as integers, are the same as the values of `np.array([0,1,0,1])`. Below, we use `==` to verify their equality. When applied to two arrays, the `==` operation is applied elementwise.

```
In [68]: np.all(keep_rows == np.array([0,1,0,1]))
```

```
Out[68]: True
```

(Here, the function `np.all()` has checked whether all entries of an array are `True`. A similar function, `np.any()`, can be used to check whether any entries of an array are `True`.)

`np.all()`
`np.any()`

However, even though `np.array([0,1,0,1])` and `keep_rows` are equal according to `==`, they index different sets of rows! The former retrieves the first, second, first, and second rows of `A`.

```
In [69]: A[np.array([0,1,0,1])]
```

```
Out[69]: array([[0, 1, 2, 3],
               [4, 5, 6, 7],
               [0, 1, 2, 3],
               [4, 5, 6, 7]])
```

By contrast, `keep_rows` retrieves only the second and fourth rows of `A` — i.e. the rows for which the Boolean equals `TRUE`.

```
In [70]: A[keep_rows]
```

```
Out[70]: array([[ 4,  5,  6,  7],
               [12, 13, 14, 15]])
```

This example shows that Booleans and integers are treated differently by `numpy`.

We again make use of the `np.ix_()` function to create a mesh containing the second and fourth rows, and the first, third, and fourth columns. This time, we apply the function to Booleans, rather than lists.

```
In [71]: keep_cols = np.zeros(A.shape[1], bool)
keep_cols[[0, 2, 3]] = True
idx_bool = np.ix_(keep_rows, keep_cols)
A[idx_bool]
```

```
Out[71]: array([[ 4,  6,  7],
               [12, 14, 15]])
```

We can also mix a list with an array of Booleans in the arguments to `np.ix_()`:

```
In [72]: idx_mixed = np.ix_([1,3], keep_cols)
A[idx_mixed]
```

```
Out[72]: array([[ 4,  6,  7],
               [12, 14, 15]])
```

For more details on indexing in `numpy`, readers are referred to the `numpy` tutorial mentioned earlier.

2.3.7 Loading Data

Data sets often contain different types of data, and may have names associated with the rows or columns. For these reasons, they typically are best accommodated using a *data frame*. We can think of a data frame as a sequence of arrays of identical length; these are the columns. Entries in the different arrays can be combined to form a row. The `pandas` library can be used to create and work with data frame objects.

data frame

Reading in a Data Set

The first step of most analyses involves importing a data set into `Python`. Before attempting to load a data set, we must make sure that `Python` knows where to find the file containing it. If the file is in the same location as this notebook file, then we are all set. Otherwise, the command `os.chdir()` can be used to *change directory*. (You will need to call `import os` before calling `os.chdir()`.)

`os.chdir()`

We will begin by reading in `Auto.csv`, available on the book website. This is a comma-separated file, and can be read in using `pd.read_csv()`:

`pd.read_csv()`

```
In [73]: import pandas as pd
Auto = pd.read_csv('Auto.csv')
Auto
```

The book website also has a whitespace-delimited version of this data, called `Auto.data`. This can be read in as follows:

```
In [74]: Auto = pd.read_csv('Auto.data', delim_whitespace=True)
```

Both `Auto.csv` and `Auto.data` are simply text files. Before loading data into `Python`, it is a good idea to view it using a text editor or other software, such as Microsoft Excel.

We now take a look at the column of `Auto` corresponding to the variable `horsepower`:

```
In [75]: Auto['horsepower']
```

```
Out[75]: 0      130.0
1      165.0
2      150.0
3      150.0
4      140.0
...
392    86.00
393    52.00
394    84.00
395    79.00
396    82.00
Name: horsepower, Length: 397, dtype: object
```

We see that the `dtype` of this column is `object`. It turns out that all values of the `horsepower` column were interpreted as strings when reading in the data. We can find out why by looking at the unique values.

```
In [76]: np.unique(Auto['horsepower'])
```

To save space, we have omitted the output of the previous code block. We see the culprit is the value `?`, which is being used to encode missing values.

To fix the problem, we must provide `pd.read_csv()` with an argument called `na_values`. Now, each instance of `?` in the file is replaced with the value `np.nan`, which means *not a number*:

```
In [77]: Auto = pd.read_csv('Auto.data',
                           na_values=['?'],
                           delim_whitespace=True)
Auto['horsepower'].sum()
```

```
Out[77]: 40952.0
```

The `Auto.shape` attribute tells us that the data has 397 observations, or rows, and nine variables, or columns.

```
In [78]: Auto.shape
```

```
Out[78]: (397, 9)
```

There are various ways to deal with missing data. In this case, since only five of the rows contain missing observations, we choose to use the `Auto.dropna()` method to simply remove these rows.

```
In [79]: Auto_new = Auto.dropna()
Auto_new.shape
```

`.dropna()`

```
Out[79]: (392, 9)
```

Basics of Selecting Rows and Columns

We can use `Auto.columns` to check the variable names.

```
In [80]: Auto = Auto_new # overwrite the previous value
Auto.columns
```

```
Out[80]: Index(['mpg', 'cylinders', 'displacement', 'horsepower',
               'weight', 'acceleration', 'year', 'origin', 'name'],
              dtype='object')
```

Accessing the rows and columns of a data frame is similar, but not identical, to accessing the rows and columns of an array. Recall that the first argument to the `[]` method is always applied to the rows of the array. Similarly, passing in a slice to the `[]` method creates a data frame whose *rows* are determined by the slice:

```
In [81]: Auto[:3]
```

```
Out[81]:
```

	mpg	cylinders	displacement	horsepower	weight	...
0	18.0	8	307.0	130.0	3504.0	...
1	15.0	8	350.0	165.0	3693.0	...
2	18.0	8	318.0	150.0	3436.0	...

Similarly, an array of Booleans can be used to subset the rows:

```
In [82]: idx_80 = Auto['year'] > 80
         Auto[idx_80]
```

However, if we pass in a list of strings to the `[]` method, then we obtain a data frame containing the corresponding set of *columns*.

```
In [83]: Auto[['mpg', 'horsepower']]
```

```
Out [83]:      mpg      horsepower
0         18.0         130.0
1         15.0         165.0
2         18.0         150.0
3         16.0         150.0
4         17.0         140.0
...      ...      ...
392        27.0         86.0
393        44.0         52.0
394        32.0         84.0
395        28.0         79.0
396        31.0         82.0
392 rows x 2 columns
```

Since we did not specify an *index* column when we loaded our data frame, the rows are labeled using integers 0 to 396.

```
In [84]: Auto.index
```

```
Out [84]: Int64Index([ 0,   1,   2,   3,   4,   5,   6,   7,   8,   9,
                    ...
                    387, 388, 389, 390, 391, 392, 393, 394, 395, 396],
                    dtype='int64', length=392)
```

We can use the `set_index()` method to re-name the rows using the contents of `Auto['name']`. `.set_index()`

```
In [85]: Auto_re = Auto.set_index('name')
         Auto_re
```

```
Out [85]:      name      mpg  cylinders  displacement  ...
chevrolet chevelle malibu  18.0          8         307.0  ...
buick skylark 32          15.0          8         350.0  ...
plymouth satellite        18.0          8         318.0  ...
amc rebel sst             16.0          8         304.0  ...
```

```
In [86]: Auto_re.columns
```

```
Out [86]: Index(['mpg', 'cylinders', 'displacement', 'horsepower',
                'weight', 'acceleration', 'year', 'origin'],
                dtype='object')
```

We see that the column `'name'` is no longer there.

Now that the index has been set to `name`, we can access rows of the data frame by `name` using the `loc[]` method of `Auto`:

`.loc[]`

```
In [87]: rows = ['amc rebel sst', 'ford torino']
Auto_re.loc[rows]
```

```
Out [87]:
```

	name	mpg	cylinders	displacement	horsepower	...
	amc rebel sst	16.0	8	304.0	150.0	...
	ford torino	17.0	8	302.0	140.0	...

As an alternative to using the index name, we could retrieve the 4th and 5th rows of `Auto` using the `iloc[]` method:

```
In [88]: Auto_re.iloc[[3,4]]
```

We can also use it to retrieve the 1st, 3rd and 4th columns of `Auto_re`:

```
In [89]: Auto_re.iloc[:, [0,2,3]]
```

We can extract the 4th and 5th rows, as well as the 1st, 3rd and 4th columns, using a single call to `iloc[]`:

```
In [90]: Auto_re.iloc[[3,4], [0,2,3]]
```

```
Out [90]:
```

	name	mpg	displacement	horsepower
	amc rebel sst	16.0	304.0	150.0
	ford torino	17.0	302.0	140.0

Index entries need not be unique: there are several cars in the data frame named `ford galaxie 500`.

```
In [91]: Auto_re.loc['ford galaxie 500', ['mpg', 'origin']]
```

```
Out [91]:
```

	name	mpg	origin
	ford galaxie 500	15.0	1
	ford galaxie 500	14.0	1
	ford galaxie 500	14.0	1

More on Selecting Rows and Columns

Suppose now that we want to create a data frame consisting of the `weight` and `origin` of the subset of cars with `year` greater than 80 — i.e. those built after 1980. To do this, we first create a Boolean array that indexes the rows. The `loc[]` method allows for Boolean entries as well as strings:

```
In [92]: idx_80 = Auto_re['year'] > 80
Auto_re.loc[idx_80, ['weight', 'origin']]
```

To do this more concisely, we can use an anonymous function called a `lambda`:

```
In [93]: Auto_re.loc[lambda df: df['year'] > 80, ['weight', 'origin']]
```

The `lambda` call creates a function that takes a single argument, here `df`, and returns `df['year']>80`. Since it is created inside the `loc[]` method for

the dataframe `Auto_re`, that dataframe will be the argument supplied. As another example of using a `lambda`, suppose that we want all cars built after 1980 that achieve greater than 30 miles per gallon:

```
In [94]: Auto_re.loc[lambda df: (df['year'] > 80) & (df['mpg'] > 30),
                  ['weight', 'origin']]
```

The symbol `&` computes an element-wise *and* operation. As another example, suppose that we want to retrieve all `Ford` and `Datsun` cars with `displacement` less than 300. We check whether each `name` entry contains either the string `ford` or `datsun` using the `str.contains()` method of the `index` attribute of the dataframe:

```
In [95]: Auto_re.loc[lambda df: (df['displacement'] < 300)
                  & (df.index.str.contains('ford')
                   | df.index.str.contains('datsun')),
                  ['weight', 'origin']]
```

`.str.
contains()`

Here, the symbol `|` computes an element-wise *or* operation.

In summary, a powerful set of operations is available to index the rows and columns of data frames. For integer based queries, use the `iloc[]` method. For string and Boolean selections, use the `loc[]` method. For functional queries that filter rows, use the `loc[]` method with a function (typically a `lambda`) in the rows argument.

2.3.8 For Loops

A `for` loop is a standard tool in many languages that repeatedly evaluates some chunk of code while varying different values inside the code. For example, suppose we loop over elements of a list and compute their sum.

```
In [96]: total = 0
         for value in [3,2,19]:
             total += value
         print('Total is: {}'.format(total))
```

Total is: 24

The indented code beneath the line with the `for` statement is run for each value in the sequence specified in the `for` statement. The loop ends either when the cell ends or when code is indented at the same level as the original `for` statement. We see that the final line above which prints the total is executed only once after the for loop has terminated. Loops can be nested by additional indentation.

```
In [97]: total = 0
         for value in [2,3,19]:
             for weight in [3, 2, 1]:
                 total += value * weight
         print('Total is: {}'.format(total))
```

Total is: 144


```
Column "food" has 16.54% missing values
Column "bar" has 25.98% missing values
Column "pickle" has 29.13% missing values
Column "snack" has 21.26% missing values
Column "popcorn" has 22.83% missing values
```

We see that the `template.format()` method expects two arguments `{0}` and `{1:.2%}`, and the latter includes some formatting information. In particular, it specifies that the second argument should be expressed as a percent with two decimal digits.

The reference docs.python.org/3/library/string.html includes many helpful and more complex examples.

2.3.9 Additional Graphical and Numerical Summaries

We can use the `ax.plot()` or `ax.scatter()` functions to display the quantitative variables. However, simply typing the variable names will produce an error message, because `Python` does not know to look in the `Auto` data set for those variables.

```
In [101]: fig, ax = subplots(figsize=(8, 8))
          ax.plot(horsepower, mpg, 'o');
```

```
NameError: name 'horsepower' is not defined
```

We can address this by accessing the columns directly:

```
In [102]: fig, ax = subplots(figsize=(8, 8))
          ax.plot(Auto['horsepower'], Auto['mpg'], 'o');
```

Alternatively, we can use the `plot()` method with the call `Auto.plot()`. Using this method, the variables can be accessed by name. The plot methods of a data frame return a familiar object: an axes. We can use it to update the plot as we did previously:

```
In [103]: ax = Auto.plot.scatter('horsepower', 'mpg');
          ax.set_title('Horsepower vs. MPG')
```

If we want to save the figure that contains a given axes, we can find the relevant figure by accessing the `figure` attribute:

```
In [104]: fig = ax.figure
          fig.savefig('horsepower_mpg.png');
```

We can further instruct the data frame to plot to a particular axes object. In this case the corresponding `plot()` method will return the modified axes we passed in as an argument. Note that when we request a one-dimensional grid of plots, the object `axes` is similarly one-dimensional. We place our scatter plot in the middle plot of a row of three plots within a figure.

```
In [105]: fig, axes = subplots(ncols=3, figsize=(15, 5))
          Auto.plot.scatter('horsepower', 'mpg', ax=axes[1]);
```

Note also that the columns of a data frame can be accessed as attributes: try typing in `Auto.horsepower`.

We now consider the `cylinders` variable. Typing in `Auto.cylinders.dtype` reveals that it is being treated as a quantitative variable. However, since there is only a small number of possible values for this variable, we may wish to treat it as qualitative. Below, we replace the `cylinders` column with a categorical version of `Auto.cylinders`. The function `pd.Series()` owes its name to the fact that `pandas` is often used in time series applications.

`pd.Series()`

```
In [106]: Auto.cylinders = pd.Series(Auto.cylinders, dtype='category')
Auto.cylinders.dtype
```

Now that `cylinders` is qualitative, we can display it using the `boxplot()` method.

`.boxplot()`

```
In [107]: fig, ax = subplots(figsize=(8, 8))
Auto.boxplot('mpg', by='cylinders', ax=ax);
```

The `hist()` method can be used to plot a *histogram*.

`.hist()`

```
In [108]: fig, ax = subplots(figsize=(8, 8))
Auto.hist('mpg', ax=ax);
```

The color of the bars and the number of bins can be changed:

```
In [109]: fig, ax = subplots(figsize=(8, 8))
Auto.hist('mpg', color='red', bins=12, ax=ax);
```

See `Auto.hist?` for more plotting options.

We can use the `pd.plotting.scatter_matrix()` function to create a *scatterplot matrix* to visualize all of the pairwise relationships between the columns in a data frame.

`pd.plotting.
scatter_
matrix()`

```
In [110]: pd.plotting.scatter_matrix(Auto);
```

We can also produce scatterplots for a subset of the variables.

```
In [111]: pd.plotting.scatter_matrix(Auto[['mpg',
                                           'displacement',
                                           'weight']]);
```

The `describe()` method produces a numerical summary of each column in a data frame.

`.describe()`

```
In [112]: Auto[['mpg', 'weight']].describe()
```

We can also produce a summary of just a single column.

```
In [113]: Auto['cylinders'].describe()
Auto['mpg'].describe()
```

To exit Jupyter, select `File / Close and Halt`.

2.4 Exercises

Conceptual

1. For each of parts (a) through (d), indicate whether we would generally expect the performance of a flexible statistical learning method to be better or worse than an inflexible method. Justify your answer.
 - (a) The sample size n is extremely large, and the number of predictors p is small.
 - (b) The number of predictors p is extremely large, and the number of observations n is small.
 - (c) The relationship between the predictors and response is highly non-linear.
 - (d) The variance of the error terms, i.e. $\sigma^2 = \text{Var}(\epsilon)$, is extremely high.
2. Explain whether each scenario is a classification or regression problem, and indicate whether we are most interested in inference or prediction. Finally, provide n and p .
 - (a) We collect a set of data on the top 500 firms in the US. For each firm we record profit, number of employees, industry and the CEO salary. We are interested in understanding which factors affect CEO salary.
 - (b) We are considering launching a new product and wish to know whether it will be a *success* or a *failure*. We collect data on 20 similar products that were previously launched. For each product we have recorded whether it was a success or failure, price charged for the product, marketing budget, competition price, and ten other variables.
 - (c) We are interested in predicting the % change in the USD/Euro exchange rate in relation to the weekly changes in the world stock markets. Hence we collect weekly data for all of 2012. For each week we record the % change in the USD/Euro, the % change in the US market, the % change in the British market, and the % change in the German market.
3. We now revisit the bias-variance decomposition.
 - (a) Provide a sketch of typical (squared) bias, variance, training error, test error, and Bayes (or irreducible) error curves, on a single plot, as we go from less flexible statistical learning methods towards more flexible approaches. The x -axis should represent the amount of flexibility in the method, and the y -axis should represent the values for each curve. There should be five curves. Make sure to label each one.
 - (b) Explain why each of the five curves has the shape displayed in part (a).

4. You will now think of some real-life applications for statistical learning.
 - (a) Describe three real-life applications in which *classification* might be useful. Describe the response, as well as the predictors. Is the goal of each application inference or prediction? Explain your answer.
 - (b) Describe three real-life applications in which *regression* might be useful. Describe the response, as well as the predictors. Is the goal of each application inference or prediction? Explain your answer.
 - (c) Describe three real-life applications in which *cluster analysis* might be useful.
5. What are the advantages and disadvantages of a very flexible (versus a less flexible) approach for regression or classification? Under what circumstances might a more flexible approach be preferred to a less flexible approach? When might a less flexible approach be preferred?
6. Describe the differences between a parametric and a non-parametric statistical learning approach. What are the advantages of a parametric approach to regression or classification (as opposed to a non-parametric approach)? What are its disadvantages?
7. The table below provides a training data set containing six observations, three predictors, and one qualitative response variable.

Obs.	X_1	X_2	X_3	Y
1	0	3	0	Red
2	2	0	0	Red
3	0	1	3	Red
4	0	1	2	Green
5	-1	0	1	Green
6	1	1	1	Red

Suppose we wish to use this data set to make a prediction for Y when $X_1 = X_2 = X_3 = 0$ using K -nearest neighbors.

- (a) Compute the Euclidean distance between each observation and the test point, $X_1 = X_2 = X_3 = 0$.
- (b) What is our prediction with $K = 1$? Why?
- (c) What is our prediction with $K = 3$? Why?
- (d) If the Bayes decision boundary in this problem is highly non-linear, then would we expect the *best* value for K to be large or small? Why?

Applied

8. This exercise relates to the `College` data set, which can be found in the file `College.csv` on the book website. It contains a number of variables for 777 different universities and colleges in the US. The variables are

- `Private` : Public/private indicator
- `Apps` : Number of applications received
- `Accept` : Number of applicants accepted
- `Enroll` : Number of new students enrolled
- `Top10perc` : New students from top 10 % of high school class
- `Top25perc` : New students from top 25 % of high school class
- `F.Undergrad` : Number of full-time undergraduates
- `P.Undergrad` : Number of part-time undergraduates
- `Outstate` : Out-of-state tuition
- `Room.Board` : Room and board costs
- `Books` : Estimated book costs
- `Personal` : Estimated personal spending
- `PhD` : Percent of faculty with Ph.D.s
- `Terminal` : Percent of faculty with terminal degree
- `S.F.Ratio` : Student/faculty ratio
- `perc.alumni` : Percent of alumni who donate
- `Expend` : Instructional expenditure per student
- `Grad.Rate` : Graduation rate

Before reading the data into `Python`, it can be viewed in Excel or a text editor.

- (a) Use the `pd.read_csv()` function to read the data into `Python`. Call the loaded data `college`. Make sure that you have the directory set to the correct location for the data.
- (b) Look at the data used in the notebook by creating and running a new cell with just the code `college` in it. You should notice that the first column is just the name of each university in a column named something like `Unnamed: 0`. We don't really want `pandas` to treat this as data. However, it may be handy to have these names for later. Try the following commands and similarly look at the resulting data frames:

```
college2 = pd.read_csv('College.csv', index_col=0)
college3 = college.rename({'Unnamed: 0': 'College'},
                          axis=1)
college3 = college3.set_index('College')
```

This has used the first column in the file as an `index` for the data frame. This means that `pandas` has given each row a name corresponding to the appropriate university. Now you should see that the first data column is `Private`. Note that the names of the colleges appear on the left of the table. We also introduced a new python object above: a *dictionary*, which is specified by `(key, value)` pairs. Keep your modified version of the data with the following:

dictionary

```
college = college3
```

- (c) Use the `describe()` method of to produce a numerical summary of the variables in the data set.
- (d) Use the `pd.plotting.scatter_matrix()` function to produce a scatterplot matrix of the first columns `[Top10perc, Apps, Enroll]`. Recall that you can reference a list `C` of columns of a data frame `A` using `A[C]`.
- (e) Use the `boxplot()` method of `college` to produce side-by-side boxplots of `Outstate` versus `Private`.
- (f) Create a new qualitative variable, called `Elite`, by *binning* the `Top10perc` variable into two groups based on whether or not the proportion of students coming from the top 10% of their high school classes exceeds 50%.

```
college['Elite'] = pd.cut(college['Top10perc'],
                          [0,0.5,1],
                          labels=['No', 'Yes'])
```

Use the `value_counts()` method of `college['Elite']` to see how many elite universities there are. Finally, use the `boxplot()` method again to produce side-by-side boxplots of `Outstate` versus `Elite`.

- (g) Use the `plot.hist()` method of `college` to produce some histograms with differing numbers of bins for a few of the quantitative variables. The command `plt.subplots(2, 2)` may be useful: it will divide the plot window into four regions so that four plots can be made simultaneously. By changing the arguments you can divide the screen up in other combinations.
 - (h) Continue exploring the data, and provide a brief summary of what you discover.
9. This exercise involves the `Auto` data set studied in the lab. Make sure that the missing values have been removed from the data.
- (a) Which of the predictors are quantitative, and which are qualitative?
 - (b) What is the *range* of each quantitative predictor? You can answer this using the `min()` and `max()` methods in `numpy`. `.min()`
`.max()`
 - (c) What is the mean and standard deviation of each quantitative predictor?

- (d) Now remove the 10th through 85th observations. What is the range, mean, and standard deviation of each predictor in the subset of the data that remains?
 - (e) Using the full data set, investigate the predictors graphically, using scatterplots or other tools of your choice. Create some plots highlighting the relationships among the predictors. Comment on your findings.
 - (f) Suppose that we wish to predict gas mileage (`mpg`) on the basis of the other variables. Do your plots suggest that any of the other variables might be useful in predicting `mpg`? Justify your answer.
10. This exercise involves the `Boston` housing data set.
- (a) To begin, load in the `Boston` data set, which is part of the `ISLP` library.
 - (b) How many rows are in this data set? How many columns? What do the rows and columns represent?
 - (c) Make some pairwise scatterplots of the predictors (columns) in this data set. Describe your findings.
 - (d) Are any of the predictors associated with per capita crime rate? If so, explain the relationship.
 - (e) Do any of the suburbs of Boston appear to have particularly high crime rates? Tax rates? Pupil-teacher ratios? Comment on the range of each predictor.
 - (f) How many of the suburbs in this data set bound the Charles river?
 - (g) What is the median pupil-teacher ratio among the towns in this data set?
 - (h) Which suburb of Boston has lowest median value of owner-occupied homes? What are the values of the other predictors for that suburb, and how do those values compare to the overall ranges for those predictors? Comment on your findings.
 - (i) In this data set, how many of the suburbs average more than seven rooms per dwelling? More than eight rooms per dwelling? Comment on the suburbs that average more than eight rooms per dwelling.



This chapter is about *linear regression*, a very simple approach for supervised learning. In particular, linear regression is a useful tool for predicting a quantitative response. It has been around for a long time and is the topic of innumerable textbooks. Though it may seem somewhat dull compared to some of the more modern statistical learning approaches described in later chapters of this book, linear regression is still a useful and widely used statistical learning method. Moreover, it serves as a good jumping-off point for newer approaches: as we will see in later chapters, many fancy statistical learning approaches can be seen as generalizations or extensions of linear regression. Consequently, the importance of having a good understanding of linear regression before studying more complex learning methods cannot be overstated. In this chapter, we review some of the key ideas underlying the linear regression model, as well as the least squares approach that is most commonly used to fit this model.

Recall the **Advertising** data from Chapter 2. Figure 2.1 displays **sales** (in thousands of units) for a particular product as a function of advertising budgets (in thousands of dollars) for **TV**, **radio**, and **newspaper** media. Suppose that in our role as statistical consultants we are asked to suggest, on the basis of this data, a marketing plan for next year that will result in high product sales. What information would be useful in order to provide such a recommendation? Here are a few important questions that we might seek to address:

1. *Is there a relationship between advertising budget and sales?*

Our first goal should be to determine whether the data provide evidence of an association between advertising expenditure and sales. If the evidence is weak, then one might argue that no money should be spent on advertising!

2. *How strong is the relationship between advertising budget and sales?*
Assuming that there is a relationship between advertising and sales, we would like to know the strength of this relationship. Does knowledge of the advertising budget provide a lot of information about product sales?
3. *Which media are associated with sales?*
Are all three media—TV, radio, and newspaper—associated with sales, or are just one or two of the media associated? To answer this question, we must find a way to separate out the individual contribution of each medium to sales when we have spent money on all three media.
4. *How large is the association between each medium and sales?*
For every dollar spent on advertising in a particular medium, by what amount will sales increase? How accurately can we predict this amount of increase?
5. *How accurately can we predict future sales?*
For any given level of television, radio, or newspaper advertising, what is our prediction for sales, and what is the accuracy of this prediction?
6. *Is the relationship linear?*
If there is approximately a straight-line relationship between advertising expenditure in the various media and sales, then linear regression is an appropriate tool. If not, then it may still be possible to transform the predictor or the response so that linear regression can be used.
7. *Is there synergy among the advertising media?*
Perhaps spending \$50,000 on television advertising and \$50,000 on radio advertising is associated with higher sales than allocating \$100,000 to either television or radio individually. In marketing, this is known as a *synergy* effect, while in statistics it is called an *interaction* effect.

synergy
interaction

It turns out that linear regression can be used to answer each of these questions. We will first discuss all of these questions in a general context, and then return to them in this specific context in Section 3.4.

3.1 Simple Linear Regression

Simple linear regression lives up to its name: it is a very straightforward approach for predicting a quantitative response Y on the basis of a single predictor variable X . It assumes that there is approximately a linear relationship between X and Y . Mathematically, we can write this linear relationship as

simple linear
regression

$$Y \approx \beta_0 + \beta_1 X. \quad (3.1)$$

You might read “ $\approx$ ” as “*is approximately modeled as*”. We will sometimes describe (3.1) by saying that we are *regressing Y on X* (or *Y onto X*).

For example, X may represent **TV** advertising and Y may represent **sales**. Then we can regress **sales** onto **TV** by fitting the model

$$\text{sales} \approx \beta_0 + \beta_1 \times \text{TV}.$$

In Equation 3.1, β_0 and β_1 are two unknown constants that represent the *intercept* and *slope* terms in the linear model. Together, β_0 and β_1 are known as the model *coefficients* or *parameters*. Once we have used our training data to produce estimates $\hat{\beta}_0$ and $\hat{\beta}_1$ for the model coefficients, we can predict future sales on the basis of a particular value of TV advertising by computing

intercept
slope
coefficient
parameter

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x, \quad (3.2)$$

where $\hat{y}$ indicates a prediction of Y on the basis of $X = x$. Here we use a *hat* symbol, $\hat{}$, to denote the estimated value for an unknown parameter or coefficient, or to denote the predicted value of the response.

3.1.1 Estimating the Coefficients

In practice, β_0 and β_1 are unknown. So before we can use (3.1) to make predictions, we must use data to estimate the coefficients. Let

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

represent n observation pairs, each of which consists of a measurement of X and a measurement of Y . In the **Advertising** example, this data set consists of the TV advertising budget and product sales in $n = 200$ different markets. (Recall that the data are displayed in Figure 2.1.) Our goal is to obtain coefficient estimates $\hat{\beta}_0$ and $\hat{\beta}_1$ such that the linear model (3.1) fits the available data well—that is, so that $y_i \approx \hat{\beta}_0 + \hat{\beta}_1 x_i$ for $i = 1, \dots, n$. In other words, we want to find an intercept $\hat{\beta}_0$ and a slope $\hat{\beta}_1$ such that the resulting line is as close as possible to the $n = 200$ data points. There are a number of ways of measuring *closeness*. However, by far the most common approach involves minimizing the *least squares* criterion, and we take that approach in this chapter. Alternative approaches will be considered in Chapter 6.

least squares

Let $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$ be the prediction for Y based on the i th value of X . Then $e_i = y_i - \hat{y}_i$ represents the i th *residual*—this is the difference between the i th observed response value and the i th response value that is predicted by our linear model. We define the *residual sum of squares* (RSS) as

residual

$$\text{RSS} = e_1^2 + e_2^2 + \dots + e_n^2,$$

residual sum
of squares

or equivalently as

$$\text{RSS} = (y_1 - \hat{\beta}_0 - \hat{\beta}_1 x_1)^2 + (y_2 - \hat{\beta}_0 - \hat{\beta}_1 x_2)^2 + \dots + (y_n - \hat{\beta}_0 - \hat{\beta}_1 x_n)^2. \quad (3.3)$$

The least squares approach chooses $\hat{\beta}_0$ and $\hat{\beta}_1$ to minimize the RSS. Using some calculus, one can show that the minimizers are

$$\begin{aligned} \hat{\beta}_1 &= \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \\ \hat{\beta}_0 &= \bar{y} - \hat{\beta}_1 \bar{x}, \end{aligned} \quad (3.4)$$

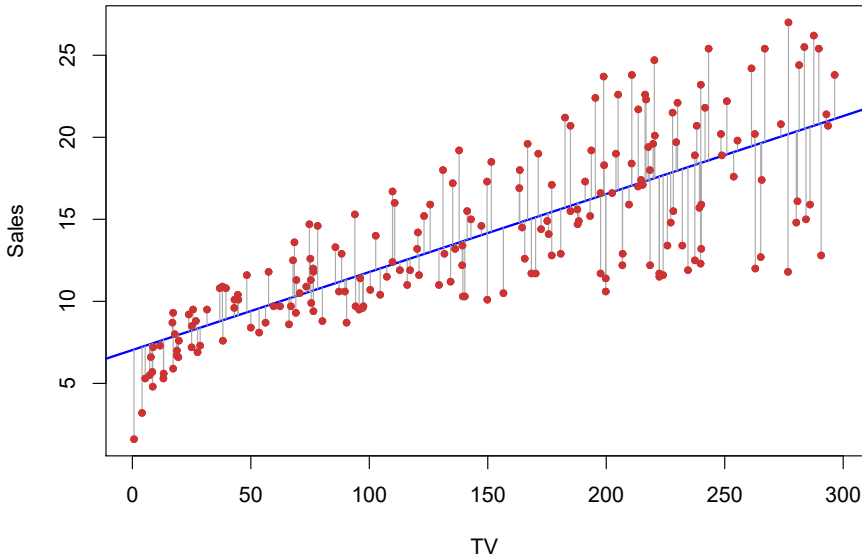


FIGURE 3.1. For the **Advertising** data, the least squares fit for the regression of **sales** onto **TV** is shown. The fit is found by minimizing the residual sum of squares. Each grey line segment represents a residual. In this case a linear fit captures the essence of the relationship, although it overestimates the trend in the left of the plot.

where $\bar{y} \equiv \frac{1}{n} \sum_{i=1}^n y_i$ and $\bar{x} \equiv \frac{1}{n} \sum_{i=1}^n x_i$ are the sample means. In other words, (3.4) defines the *least squares coefficient estimates* for simple linear regression.

Figure 3.1 displays the simple linear regression fit to the **Advertising** data, where $\hat{\beta}_0 = 7.03$ and $\hat{\beta}_1 = 0.0475$. In other words, according to this approximation, an additional \$1,000 spent on TV advertising is associated with selling approximately 47.5 additional units of the product. In Figure 3.2, we have computed RSS for a number of values of β_0 and β_1 , using the advertising data with **sales** as the response and **TV** as the predictor. In each plot, the red dot represents the pair of least squares estimates $(\hat{\beta}_0, \hat{\beta}_1)$ given by (3.4). These values clearly minimize the RSS.

3.1.2 Assessing the Accuracy of the Coefficient Estimates

Recall from (2.1) that we assume that the *true* relationship between X and Y takes the form $Y = f(X) + \epsilon$ for some unknown function f , where ϵ is a mean-zero random error term. If f is to be approximated by a linear function, then we can write this relationship as

$$Y = \beta_0 + \beta_1 X + \epsilon. \quad (3.5)$$

Here β_0 is the intercept term—that is, the expected value of Y when $X = 0$, and β_1 is the slope—the average increase in Y associated with a one-unit increase in X . The error term is a catch-all for what we miss with this simple model: the true relationship is probably not linear, there may be other variables that cause variation in Y , and there may be measurement error. We typically assume that the error term is independent of X .

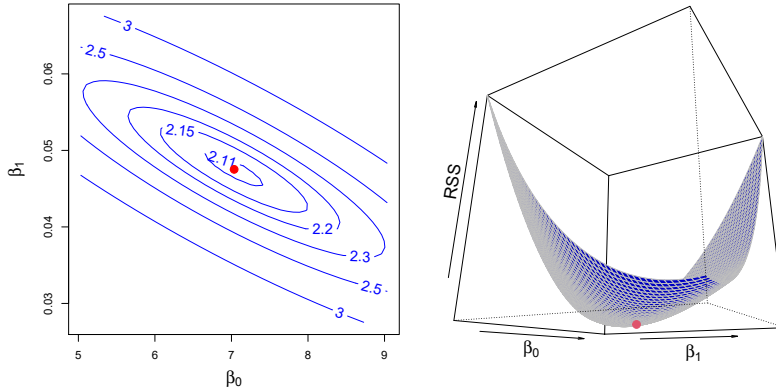


FIGURE 3.2. Contour and three-dimensional plots of the RSS on the **Advertising** data, using **sales** as the response and **TV** as the predictor. The red dots correspond to the least squares estimates $\hat{\beta}_0$ and $\hat{\beta}_1$, given by (3.4).

The model given by (3.5) defines the *population regression line*, which is the best linear approximation to the true relationship between X and Y .¹ The least squares regression coefficient estimates (3.4) characterize the *least squares line* (3.2). The left-hand panel of Figure 3.3 displays these two lines in a simple simulated example. We created 100 random X s, and generated 100 corresponding Y s from the model

$$Y = 2 + 3X + \epsilon, \quad (3.6)$$

where ϵ was generated from a normal distribution with mean zero. The red line in the left-hand panel of Figure 3.3 displays the *true* relationship, $f(X) = 2 + 3X$, while the blue line is the least squares estimate based on the observed data. The true relationship is generally not known for real data, but the least squares line can always be computed using the coefficient estimates given in (3.4). In other words, in real applications, we have access to a set of observations from which we can compute the least squares line; however, the population regression line is unobserved. In the right-hand panel of Figure 3.3 we have generated ten different data sets from the model given by (3.6) and plotted the corresponding ten least squares lines. Notice that different data sets generated from the same true model result in slightly different least squares lines, but the unobserved population regression line does not change.

At first glance, the difference between the population regression line and the least squares line may seem subtle and confusing. We only have one data set, and so what does it mean that two different lines describe the relationship between the predictor and the response? Fundamentally, the concept of these two lines is a natural extension of the standard statistical approach of using information from a sample to estimate characteristics of a large population. For example, suppose that we are interested in knowing

population
regression
line

least squares
line

¹The assumption of linearity is often a useful working model. However, despite what many textbooks might tell us, we seldom believe that the true relationship is linear.

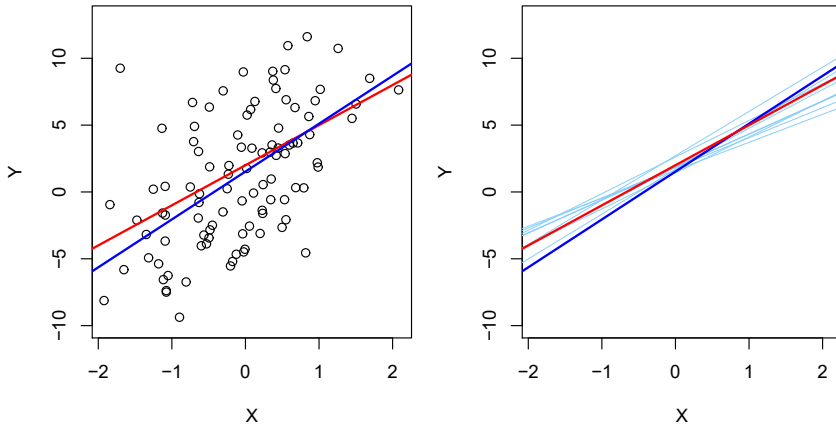


FIGURE 3.3. A simulated data set. Left: The red line represents the true relationship, $f(X) = 2 + 3X$, which is known as the population regression line. The blue line is the least squares line; it is the least squares estimate for $f(X)$ based on the observed data, shown in black. Right: The population regression line is again shown in red, and the least squares line in dark blue. In light blue, ten least squares lines are shown, each computed on the basis of a separate random set of observations. Each least squares line is different, but on average, the least squares lines are quite close to the population regression line.

the population mean μ of some random variable Y . Unfortunately, μ is unknown, but we do have access to n observations from Y , $y_1, \dots, y_n$, which we can use to estimate μ . A reasonable estimate is $\hat{\mu} = \bar{y}$, where $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$ is the sample mean. The sample mean and the population mean are different, but in general the sample mean will provide a good estimate of the population mean. In the same way, the unknown coefficients β_0 and β_1 in linear regression define the population regression line. We seek to estimate these unknown coefficients using $\hat{\beta}_0$ and $\hat{\beta}_1$ given in (3.4). These coefficient estimates define the least squares line.

The analogy between linear regression and estimation of the mean of a random variable is an apt one based on the concept of *bias*. If we use the sample mean $\hat{\mu}$ to estimate μ , this estimate is *unbiased*, in the sense that on average, we expect $\hat{\mu}$ to equal μ . What exactly does this mean? It means that on the basis of one particular set of observations $y_1, \dots, y_n$, $\hat{\mu}$ might overestimate μ , and on the basis of another set of observations, $\hat{\mu}$ might underestimate μ . But if we could average a huge number of estimates of μ obtained from a huge number of sets of observations, then this average would *exactly* equal μ . Hence, an unbiased estimator does not *systematically* over- or under-estimate the true parameter. The property of unbiasedness holds for the least squares coefficient estimates given by (3.4) as well: if we estimate β_0 and β_1 on the basis of a particular data set, then our estimates won't be exactly equal to β_0 and β_1 . But if we could average the estimates obtained over a huge number of data sets, then the average of these estimates would be spot on! In fact, we can see from the right-hand panel of Figure 3.3 that the average of many least squares lines, each

bias
unbiased

estimated from a separate data set, is pretty close to the true population regression line.

We continue the analogy with the estimation of the population mean μ of a random variable Y . A natural question is as follows: how accurate is the sample mean $\hat{\mu}$ as an estimate of μ ? We have established that the average of $\hat{\mu}$'s over many data sets will be very close to μ , but that a single estimate $\hat{\mu}$ may be a substantial underestimate or overestimate of μ . How far off will that single estimate of $\hat{\mu}$ be? In general, we answer this question by computing the *standard error* of $\hat{\mu}$, written as $\text{SE}(\hat{\mu})$. We have the well-known formula

standard
error

$$\text{Var}(\hat{\mu}) = \text{SE}(\hat{\mu})^2 = \frac{\sigma^2}{n}, \quad (3.7)$$

where σ is the standard deviation of each of the realizations y_i of Y .² Roughly speaking, the standard error tells us the average amount that this estimate $\hat{\mu}$ differs from the actual value of μ . Equation 3.7 also tells us how this deviation shrinks with n —the more observations we have, the smaller the standard error of $\hat{\mu}$. In a similar vein, we can wonder how close $\hat{\beta}_0$ and $\hat{\beta}_1$ are to the true values β_0 and β_1 . To compute the standard errors associated with $\hat{\beta}_0$ and $\hat{\beta}_1$, we use the following formulas:

$$\text{SE}(\hat{\beta}_0)^2 = \sigma^2 \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2} \right], \quad \text{SE}(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad (3.8)$$

where $\sigma^2 = \text{Var}(\epsilon)$. For these formulas to be strictly valid, we need to assume that the errors ϵ_i for each observation have common variance σ^2 and are uncorrelated. This is clearly not true in Figure 3.1, but the formula still turns out to be a good approximation. Notice in the formula that $\text{SE}(\hat{\beta}_1)$ is smaller when the x_i are more spread out; intuitively we have more *leverage* to estimate a slope when this is the case. We also see that $\text{SE}(\hat{\beta}_0)$ would be the same as $\text{SE}(\hat{\mu})$ if $\bar{x}$ were zero (in which case $\hat{\beta}_0$ would be equal to $\bar{y}$). In general, σ^2 is not known, but can be estimated from the data. This estimate of σ is known as the *residual standard error*, and is given by the formula $\text{RSE} = \sqrt{\text{RSS}/(n-2)}$. Strictly speaking, when σ^2 is estimated from the data we should write $\widehat{\text{SE}}(\hat{\beta}_1)$ to indicate that an estimate has been made, but for simplicity of notation we will drop this extra “hat”.

residual
standard
error

Standard errors can be used to compute *confidence intervals*. A 95% confidence interval is defined as a range of values such that with 95% probability, the range will contain the true unknown value of the parameter. The range is defined in terms of lower and upper limits computed from the sample of data. A 95% confidence interval has the following property: if we take repeated samples and construct the confidence interval for each sample, 95% of the intervals will contain the true unknown value of the parameter. For linear regression, the 95% confidence interval for β_1 approximately takes the form

confidence
interval

$$\hat{\beta}_1 \pm 2 \cdot \text{SE}(\hat{\beta}_1). \quad (3.9)$$

²This formula holds provided that the n observations are uncorrelated.

That is, there is approximately a 95 % chance that the interval

$$\left[\hat{\beta}_1 - 2 \cdot \text{SE}(\hat{\beta}_1), \hat{\beta}_1 + 2 \cdot \text{SE}(\hat{\beta}_1) \right] \quad (3.10)$$

will contain the true value of β_1 .³ Similarly, a confidence interval for β_0 approximately takes the form

$$\hat{\beta}_0 \pm 2 \cdot \text{SE}(\hat{\beta}_0). \quad (3.11)$$

In the case of the advertising data, the 95 % confidence interval for β_0 is [6.130, 7.935] and the 95 % confidence interval for β_1 is [0.042, 0.053]. Therefore, we can conclude that in the absence of any advertising, sales will, on average, fall somewhere between 6,130 and 7,935 units. Furthermore, for each \$1,000 increase in television advertising, there will be an average increase in sales of between 42 and 53 units.

Standard errors can also be used to perform *hypothesis tests* on the coefficients. The most common hypothesis test involves testing the *null hypothesis* of

$$H_0 : \text{There is no relationship between } X \text{ and } Y \quad (3.12)$$

versus the *alternative hypothesis*

$$H_a : \text{There is some relationship between } X \text{ and } Y. \quad (3.13)$$

Mathematically, this corresponds to testing

$$H_0 : \beta_1 = 0$$

versus

$$H_a : \beta_1 \neq 0,$$

since if $\beta_1 = 0$ then the model (3.5) reduces to $Y = \beta_0 + \epsilon$, and X is not associated with Y . To test the null hypothesis, we need to determine whether $\hat{\beta}_1$, our estimate for β_1 , is sufficiently far from zero that we can be confident that β_1 is non-zero. How far is far enough? This of course depends on the accuracy of $\hat{\beta}_1$ —that is, it depends on $\text{SE}(\hat{\beta}_1)$. If $\text{SE}(\hat{\beta}_1)$ is small, then even relatively small values of $\hat{\beta}_1$ may provide strong evidence that $\beta_1 \neq 0$, and hence that there is a relationship between X and Y . In contrast, if $\text{SE}(\hat{\beta}_1)$ is large, then $\hat{\beta}_1$ must be large in absolute value in order for us to reject the null hypothesis. In practice, we compute a *t-statistic*, given by

$$t = \frac{\hat{\beta}_1 - 0}{\text{SE}(\hat{\beta}_1)}, \quad (3.14)$$

³*Approximately* for several reasons. Equation 3.10 relies on the assumption that the errors are Gaussian. Also, the factor of 2 in front of the $\text{SE}(\hat{\beta}_1)$ term will vary slightly depending on the number of observations n in the linear regression. To be precise, rather than the number 2, (3.10) should contain the 97.5 % quantile of a t -distribution with $n-2$ degrees of freedom. Details of how to compute the 95 % confidence interval precisely in R will be provided later in this chapter.

	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	7.0325	0.4578	15.36	< 0.0001
TV	0.0475	0.0027	17.67	< 0.0001

TABLE 3.1. For the **Advertising** data, coefficients of the least squares model for the regression of number of units sold on TV advertising budget. An increase of \$1,000 in the TV advertising budget is associated with an increase in sales by around 50 units. (Recall that the **sales** variable is in thousands of units, and the **TV** variable is in thousands of dollars.)

which measures the number of standard deviations that $\hat{\beta}_1$ is away from 0. If there really is no relationship between X and Y , then we expect that (3.14) will have a t -distribution with $n - 2$ degrees of freedom. The t -distribution has a bell shape and for values of n greater than approximately 30 it is quite similar to the standard normal distribution. Consequently, it is a simple matter to compute the probability of observing any number equal to $|t|$ or larger in absolute value, assuming $\beta_1 = 0$. We call this probability the *p-value*. Roughly speaking, we interpret the *p-value* as follows: a small *p-value* indicates that it is unlikely to observe such a substantial association between the predictor and the response due to chance, in the absence of any real association between the predictor and the response. Hence, if we see a small *p-value*, then we can infer that there is an association between the predictor and the response. We *reject the null hypothesis*—that is, we declare a relationship to exist between X and Y —if the *p-value* is small enough. Typical *p-value* cutoffs for rejecting the null hypothesis are 5% or 1%, although this topic will be explored in much greater detail in Chapter 13. When $n = 30$, these correspond to t -statistics (3.14) of around 2 and 2.75, respectively.

Table 3.1 provides details of the least squares model for the regression of number of units sold on TV advertising budget for the **Advertising** data. Notice that the coefficients for $\hat{\beta}_0$ and $\hat{\beta}_1$ are very large relative to their standard errors, so the t -statistics are also large; the probabilities of seeing such values if H_0 is true are virtually zero. Hence we can conclude that $\beta_0 \neq 0$ and $\beta_1 \neq 0$.⁴

3.1.3 Assessing the Accuracy of the Model

Once we have rejected the null hypothesis (3.12) in favor of the alternative hypothesis (3.13), it is natural to want to quantify *the extent to which the model fits the data*. The quality of a linear regression fit is typically assessed using two related quantities: the *residual standard error* (RSE) and the R^2 statistic.

⁴In Table 3.1, a small *p-value* for the intercept indicates that we can reject the null hypothesis that $\beta_0 = 0$, and a small *p-value* for **TV** indicates that we can reject the null hypothesis that $\beta_1 = 0$. Rejecting the latter null hypothesis allows us to conclude that there is a relationship between **TV** and **sales**. Rejecting the former allows us to conclude that in the absence of **TV** expenditure, **sales** are non-zero.

Quantity	Value
Residual standard error	3.26
R^2	0.612
F -statistic	312.1

TABLE 3.2. For the **Advertising** data, more information about the least squares model for the regression of number of units sold on TV advertising budget.

Table 3.2 displays the RSE, the R^2 statistic, and the F -statistic (to be described in Section 3.2.2) for the linear regression of number of units sold on TV advertising budget.

Residual Standard Error

Recall from the model (3.5) that associated with each observation is an error term ϵ . Due to the presence of these error terms, even if we knew the true regression line (i.e. even if β_0 and β_1 were known), we would not be able to perfectly predict Y from X . The RSE is an estimate of the standard deviation of ϵ . Roughly speaking, it is the average amount that the response will deviate from the true regression line. It is computed using the formula

$$\text{RSE} = \sqrt{\frac{1}{n-2} \text{RSS}} = \sqrt{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (3.15)$$

Note that RSS was defined in Section 3.1.1, and is given by the formula

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (3.16)$$

In the case of the advertising data, we see from the linear regression output in Table 3.2 that the RSE is 3.26. In other words, actual sales in each market deviate from the true regression line by approximately 3,260 units, on average. Another way to think about this is that even if the model were correct and the true values of the unknown coefficients β_0 and β_1 were known exactly, any prediction of sales on the basis of TV advertising would still be off by about 3,260 units on average. Of course, whether or not 3,260 units is an acceptable prediction error depends on the problem context. In the advertising data set, the mean value of **sales** over all markets is approximately 14,000 units, and so the percentage error is $3,260/14,000 = 23\%$.

The RSE is considered a measure of the *lack of fit* of the model (3.5) to the data. If the predictions obtained using the model are very close to the true outcome values—that is, if $\hat{y}_i \approx y_i$ for $i = 1, \dots, n$ —then (3.15) will be small, and we can conclude that the model fits the data very well. On the other hand, if $\hat{y}_i$ is very far from y_i for one or more observations, then the RSE may be quite large, indicating that the model doesn't fit the data well.

R^2 Statistic

The RSE provides an absolute measure of lack of fit of the model (3.5) to the data. But since it is measured in the units of Y , it is not always

clear what constitutes a good RSE. The R^2 statistic provides an alternative measure of fit. It takes the form of a *proportion*—the proportion of variance explained—and so it always takes on a value between 0 and 1, and is independent of the scale of Y .

To calculate R^2 , we use the formula

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}} \quad (3.17)$$

where $\text{TSS} = \sum (y_i - \bar{y})^2$ is the *total sum of squares*, and RSS is defined in (3.16). TSS measures the total variance in the response Y , and can be thought of as the amount of variability inherent in the response before the regression is performed. In contrast, RSS measures the amount of variability that is left unexplained after performing the regression. Hence, $\text{TSS} - \text{RSS}$ measures the amount of variability in the response that is explained (or removed) by performing the regression, and R^2 measures the *proportion of variability in Y that can be explained using X* . An R^2 statistic that is close to 1 indicates that a large proportion of the variability in the response is explained by the regression. A number near 0 indicates that the regression does not explain much of the variability in the response; this might occur because the linear model is wrong, or the error variance σ^2 is high, or both. In Table 3.2, the R^2 was 0.61, and so just under two-thirds of the variability in **sales** is explained by a linear regression on **TV**.

total sum of
squares

The R^2 statistic (3.17) has an interpretational advantage over the RSE (3.15), since unlike the RSE, it always lies between 0 and 1. However, it can still be challenging to determine what is a *good* R^2 value, and in general, this will depend on the application. For instance, in certain problems in physics, we may know that the data truly comes from a linear model with a small residual error. In this case, we would expect to see an R^2 value that is extremely close to 1, and a substantially smaller R^2 value might indicate a serious problem with the experiment in which the data were generated. On the other hand, in typical applications in biology, psychology, marketing, and other domains, the linear model (3.5) is at best an extremely rough approximation to the data, and residual errors due to other unmeasured factors are often very large. In this setting, we would expect only a very small proportion of the variance in the response to be explained by the predictor, and an R^2 value well below 0.1 might be more realistic!

The R^2 statistic is a measure of the linear relationship between X and Y . Recall that *correlation*, defined as

correlation

$$\text{Cor}(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}, \quad (3.18)$$

is also a measure of the linear relationship between X and Y .⁵ This suggests that we might be able to use $r = \text{Cor}(X, Y)$ instead of R^2 in order to assess the fit of the linear model. In fact, it can be shown that in the simple

⁵We note that in fact, the right-hand side of (3.18) is the sample correlation; thus, it would be more correct to write $\text{Cor}(X, Y)$; however, we omit the “hat” for ease of notation.

Simple regression of sales on radio				
	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	9.312	0.563	16.54	< 0.0001
radio	0.203	0.020	9.92	< 0.0001

Simple regression of sales on newspaper				
	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	12.351	0.621	19.88	< 0.0001
newspaper	0.055	0.017	3.30	0.00115

TABLE 3.3. *More simple linear regression models for the Advertising data. Coefficients of the simple linear regression model for number of units sold on Top: radio advertising budget and Bottom: newspaper advertising budget. A \$1,000 increase in spending on radio advertising is associated with an average increase in sales by around 203 units, while the same increase in spending on newspaper advertising is associated with an average increase in sales by around 55 units. (Note that the **sales** variable is in thousands of units, and the **radio** and **newspaper** variables are in thousands of dollars.)*

linear regression setting, $R^2 = r^2$. In other words, the squared correlation and the R^2 statistic are identical. However, in the next section we will discuss the multiple linear regression problem, in which we use several predictors simultaneously to predict the response. The concept of correlation between the predictors and the response does not extend automatically to this setting, since correlation quantifies the association between a single pair of variables rather than between a larger number of variables. We will see that R^2 fills this role.

3.2 Multiple Linear Regression

Simple linear regression is a useful approach for predicting a response on the basis of a single predictor variable. However, in practice we often have more than one predictor. For example, in the Advertising data, we have examined the relationship between sales and TV advertising. We also have data for the amount of money spent advertising on the radio and in newspapers, and we may want to know whether either of these two media is associated with sales. How can we extend our analysis of the advertising data in order to accommodate these two additional predictors?

One option is to run three separate simple linear regressions, each of which uses a different advertising medium as a predictor. For instance, we can fit a simple linear regression to predict sales on the basis of the amount spent on radio advertisements. Results are shown in Table 3.3 (top table). We find that a \$1,000 increase in spending on radio advertising is associated with an increase in sales of around 203 units. Table 3.3 (bottom table) contains the least squares coefficients for a simple linear regression of sales onto newspaper advertising budget. A \$1,000 increase in newspaper advertising budget is associated with an increase in sales of approximately 55 units.

However, the approach of fitting a separate simple linear regression model for each predictor is not entirely satisfactory. First of all, it is unclear how to make a single prediction of sales given the three advertising media budgets, since each of the budgets is associated with a separate regression equation. Second, each of the three regression equations ignores the other two media in forming estimates for the regression coefficients. We will see shortly that if the media budgets are correlated with each other in the 200 markets in our data set, then this can lead to very misleading estimates of the association between each media budget and sales.

Instead of fitting a separate simple linear regression model for each predictor, a better approach is to extend the simple linear regression model (3.5) so that it can directly accommodate multiple predictors. We can do this by giving each predictor a separate slope coefficient in a single model. In general, suppose that we have p distinct predictors. Then the multiple linear regression model takes the form

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon, \quad (3.19)$$

where X_j represents the j th predictor and β_j quantifies the association between that variable and the response. We interpret β_j as the *average* effect on Y of a one unit increase in X_j , *holding all other predictors fixed*. In the advertising example, (3.19) becomes

$$\text{sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times \text{newspaper} + \epsilon. \quad (3.20)$$

3.2.1 Estimating the Regression Coefficients

As was the case in the simple linear regression setting, the regression coefficients $\beta_0, \beta_1, \dots, \beta_p$ in (3.19) are unknown, and must be estimated. Given estimates $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$, we can make predictions using the formula

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p. \quad (3.21)$$

The parameters are estimated using the same least squares approach that we saw in the context of simple linear regression. We choose $\beta_0, \beta_1, \dots, \beta_p$ to minimize the sum of squared residuals

$$\begin{aligned} \text{RSS} &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \hat{\beta}_2 x_{i2} - \cdots - \hat{\beta}_p x_{ip})^2. \end{aligned} \quad (3.22)$$

The values $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ that minimize (3.22) are the multiple least squares regression coefficient estimates. Unlike the simple linear regression estimates given in (3.4), the multiple regression coefficient estimates have somewhat complicated forms that are most easily represented using matrix algebra. For this reason, we do not provide them here. Any statistical software package can be used to compute these coefficient estimates, and later in this chapter we will show how this can be done in R. Figure 3.4

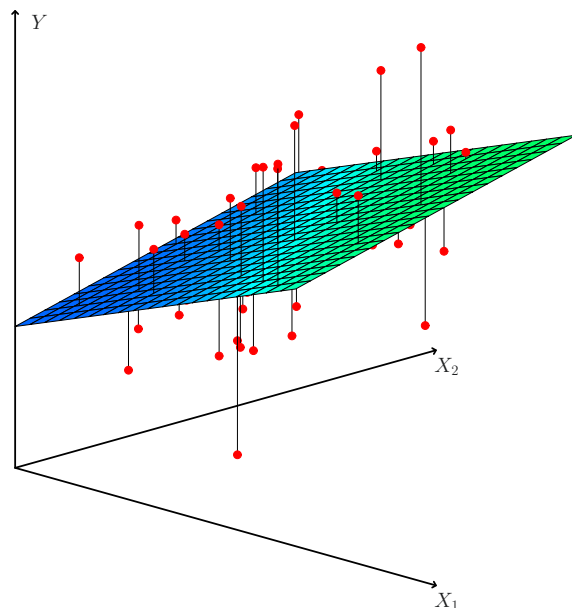


FIGURE 3.4. In a three-dimensional setting, with two predictors and one response, the least squares regression line becomes a plane. The plane is chosen to minimize the sum of the squared vertical distances between each observation (shown in red) and the plane.

illustrates an example of the least squares fit to a toy data set with $p = 2$ predictors.

Table 3.4 displays the multiple regression coefficient estimates when TV, radio, and newspaper advertising budgets are used to predict product sales using the **Advertising** data. We interpret these results as follows: for a given amount of TV and newspaper advertising, spending an additional \$1,000 on radio advertising is associated with approximately 189 units of additional sales. Comparing these coefficient estimates to those displayed in Tables 3.1 and 3.3, we notice that the multiple regression coefficient estimates for **TV** and **radio** are pretty similar to the simple linear regression coefficient estimates. However, while the **newspaper** regression coefficient estimate in Table 3.3 was significantly non-zero, the coefficient estimate for **newspaper** in the multiple regression model is close to zero, and the corresponding p -value is no longer significant, with a value around 0.86. This illustrates that the simple and multiple regression coefficients can be quite different. This difference stems from the fact that in the simple regression case, the slope term represents the average increase in product sales associated with a \$1,000 increase in newspaper advertising, ignoring other predictors such as **TV** and **radio**. By contrast, in the multiple regression setting, the coefficient for **newspaper** represents the average increase in product sales associated with increasing newspaper spending by \$1,000 while holding **TV** and **radio** fixed.

Does it make sense for the multiple regression to suggest no relationship between **sales** and **newspaper** while the simple linear regression implies the

	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	2.939	0.3119	9.42	< 0.0001
TV	0.046	0.0014	32.81	< 0.0001
radio	0.189	0.0086	21.89	< 0.0001
newspaper	−0.001	0.0059	−0.18	0.8599

TABLE 3.4. For the Advertising data, least squares coefficient estimates of the multiple linear regression of number of units sold on TV, radio, and newspaper advertising budgets.

	TV	radio	newspaper	sales
TV	1.0000	0.0548	0.0567	0.7822
radio		1.0000	0.3541	0.5762
newspaper			1.0000	0.2283
sales				1.0000

TABLE 3.5. Correlation matrix for TV, radio, newspaper, and sales for the Advertising data.

opposite? In fact it does. Consider the correlation matrix for the three predictor variables and response variable, displayed in Table 3.5. Notice that the correlation between radio and newspaper is 0.35. This indicates that markets with high newspaper advertising tend to also have high radio advertising. Now suppose that the multiple regression is correct and newspaper advertising is not associated with sales, but radio advertising is associated with sales. Then in markets where we spend more on radio our sales will tend to be higher, and as our correlation matrix shows, we also tend to spend more on newspaper advertising in those same markets. Hence, in a simple linear regression which only examines sales versus newspaper, we will observe that higher values of newspaper tend to be associated with higher values of sales, even though newspaper advertising is not directly associated with sales. So newspaper advertising is a surrogate for radio advertising; newspaper gets “credit” for the association between radio on sales.

This slightly counterintuitive result is very common in many real life situations. Consider an absurd example to illustrate the point. Running a regression of shark attacks versus ice cream sales for data collected at a given beach community over a period of time would show a positive relationship, similar to that seen between sales and newspaper. Of course no one has (yet) suggested that ice creams should be banned at beaches to reduce shark attacks. In reality, higher temperatures cause more people to visit the beach, which in turn results in more ice cream sales and more shark attacks. A multiple regression of shark attacks onto ice cream sales and temperature reveals that, as intuition implies, ice cream sales is no longer a significant predictor after adjusting for temperature.

3.2.2 Some Important Questions

When we perform multiple linear regression, we usually are interested in answering a few important questions.

1. *Is at least one of the predictors $X_1, X_2, \dots, X_p$ useful in predicting the response?*
2. *Do all the predictors help to explain Y , or is only a subset of the predictors useful?*
3. *How well does the model fit the data?*
4. *Given a set of predictor values, what response value should we predict, and how accurate is our prediction?*

We now address each of these questions in turn.

One: Is There a Relationship Between the Response and Predictors?

Recall that in the simple linear regression setting, in order to determine whether there is a relationship between the response and the predictor we can simply check whether $\beta_1 = 0$. In the multiple regression setting with p predictors, we need to ask whether all of the regression coefficients are zero, i.e. whether $\beta_1 = \beta_2 = \dots = \beta_p = 0$. As in the simple linear regression setting, we use a hypothesis test to answer this question. We test the null hypothesis,

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$$

versus the alternative

$$H_a : \text{at least one } \beta_j \text{ is non-zero.}$$

This hypothesis test is performed by computing the *F-statistic*,

F-statistic

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n - p - 1)}, \quad (3.23)$$

where, as with simple linear regression, $\text{TSS} = \sum(y_i - \bar{y})^2$ and $\text{RSS} = \sum(y_i - \hat{y}_i)^2$. If the linear model assumptions are correct, one can show that

$$E\{\text{RSS}/(n - p - 1)\} = \sigma^2$$

and that, provided H_0 is true,

$$E\{(\text{TSS} - \text{RSS})/p\} = \sigma^2.$$

Hence, when there is no relationship between the response and predictors, one would expect the *F*-statistic to take on a value close to 1. On the other hand, if H_a is true, then $E\{(\text{TSS} - \text{RSS})/p\} > \sigma^2$, so we expect *F* to be greater than 1.

The *F*-statistic for the multiple linear regression model obtained by regressing **sales** onto **radio**, **TV**, and **newspaper** is shown in Table 3.6. In this example the *F*-statistic is 570. Since this is far larger than 1, it provides compelling evidence against the null hypothesis H_0 . In other words, the large *F*-statistic suggests that at least one of the advertising media must be related to **sales**. However, what if the *F*-statistic had been closer to 1? How large does the *F*-statistic need to be before we can reject H_0 and

Quantity	Value
Residual standard error	1.69
R^2	0.897
F -statistic	570

TABLE 3.6. More information about the least squares model for the regression of number of units sold on TV, newspaper, and radio advertising budgets in the Advertising data. Other information about this model was displayed in Table 3.4.

conclude that there is a relationship? It turns out that the answer depends on the values of n and p . When n is large, an F -statistic that is just a little larger than 1 might still provide evidence against H_0 . In contrast, a larger F -statistic is needed to reject H_0 if n is small. When H_0 is true and the errors ϵ_i have a normal distribution, the F -statistic follows an F -distribution.⁶ For any given value of n and p , any statistical software package can be used to compute the p -value associated with the F -statistic using this distribution. Based on this p -value, we can determine whether or not to reject H_0 . For the advertising data, the p -value associated with the F -statistic in Table 3.6 is essentially zero, so we have extremely strong evidence that at least one of the media is associated with increased sales.

In (3.23) we are testing H_0 that all the coefficients are zero. Sometimes we want to test that a particular subset of q of the coefficients are zero. This corresponds to a null hypothesis

$$H_0 : \beta_{p-q+1} = \beta_{p-q+2} = \cdots = \beta_p = 0,$$

where for convenience we have put the variables chosen for omission at the end of the list. In this case we fit a second model that uses all the variables *except* those last q . Suppose that the residual sum of squares for that model is RSS_0 . Then the appropriate F -statistic is

$$F = \frac{(\text{RSS}_0 - \text{RSS})/q}{\text{RSS}/(n - p - 1)}. \quad (3.24)$$

Notice that in Table 3.4, for each individual predictor a t -statistic and a p -value were reported. These provide information about whether each individual predictor is related to the response, after adjusting for the other predictors. It turns out that each of these is exactly equivalent⁷ to the F -test that omits that single variable from the model, leaving all the others in—i.e. $q=1$ in (3.24). So it reports the *partial effect* of adding that variable to the model. For instance, as we discussed earlier, these p -values indicate that TV and radio are related to sales, but that there is no evidence that newspaper is associated with sales, when TV and radio are held fixed.

Given these individual p -values for each variable, why do we need to look at the overall F -statistic? After all, it seems likely that if any one of the p -values for the individual variables is very small, then *at least one of the predictors is related to the response*. However, this logic is flawed, especially when the number of predictors p is large.

⁶Even if the errors are not normally-distributed, the F -statistic approximately follows an F -distribution provided that the sample size n is large.

⁷The square of each t -statistic is the corresponding F -statistic.

For instance, consider an example in which $p = 100$ and $H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$ is true, so no variable is truly associated with the response. In this situation, about 5% of the p -values associated with each variable (of the type shown in Table 3.4) will be below 0.05 by chance. In other words, we expect to see approximately five *small* p -values even in the absence of any true association between the predictors and the response.⁸ In fact, it is likely that we will observe at least one p -value below 0.05 by chance! Hence, if we use the individual t -statistics and associated p -values in order to decide whether or not there is any association between the variables and the response, there is a very high chance that we will incorrectly conclude that there is a relationship. However, the F -statistic does not suffer from this problem because it adjusts for the number of predictors. Hence, if H_0 is true, there is only a 5% chance that the F -statistic will result in a p -value below 0.05, regardless of the number of predictors or the number of observations.

The approach of using an F -statistic to test for any association between the predictors and the response works when p is relatively small, and certainly small compared to n . However, sometimes we have a very large number of variables. If $p > n$ then there are more coefficients β_j to estimate than observations from which to estimate them. In this case we cannot even fit the multiple linear regression model using least squares, so the F -statistic cannot be used, and neither can most of the other concepts that we have seen so far in this chapter. When p is large, some of the approaches discussed in the next section, such as *forward selection*, can be used. This *high-dimensional* setting is discussed in greater detail in Chapter 6.

high-
dimensional

Two: Deciding on Important Variables

As discussed in the previous section, the first step in a multiple regression analysis is to compute the F -statistic and to examine the associated p -value. If we conclude on the basis of that p -value that at least one of the predictors is related to the response, then it is natural to wonder *which* are the guilty ones! We could look at the individual p -values as in Table 3.4, but as discussed (and as further explored in Chapter 13), if p is large we are likely to make some false discoveries.

It is possible that all of the predictors are associated with the response, but it is more often the case that the response is only associated with a subset of the predictors. The task of determining which predictors are associated with the response, in order to fit a single model involving only those predictors, is referred to as *variable selection*. The variable selection problem is studied extensively in Chapter 6, and so here we will provide only a brief outline of some classical approaches.

variable
selection

Ideally, we would like to perform variable selection by trying out a lot of different models, each containing a different subset of the predictors. For instance, if $p = 2$, then we can consider four models: (1) a model containing no variables, (2) a model containing X_1 only, (3) a model containing

⁸This is related to the important concept of *multiple testing*, which is the focus of Chapter 13.

X_2 only, and (4) a model containing both X_1 and X_2 . We can then select the *best* model out of all of the models that we have considered. How do we determine which model is best? Various statistics can be used to judge the quality of a model. These include *Mallow's C_p* , *Akaike information criterion* (AIC), *Bayesian information criterion* (BIC), and *adjusted R^2* . These are discussed in more detail in Chapter 6. We can also determine which model is best by plotting various model outputs, such as the residuals, in order to search for patterns.

Mallow's C_p
Akaike
information
criterion
Bayesian
information
criterion
adjusted R^2

Unfortunately, there are a total of 2^p models that contain subsets of p variables. This means that even for moderate p , trying out every possible subset of the predictors is infeasible. For instance, we saw that if $p = 2$, then there are $2^2 = 4$ models to consider. But if $p = 30$, then we must consider $2^{30} = 1,073,741,824$ models! This is not practical. Therefore, unless p is very small, we cannot consider all 2^p models, and instead we need an automated and efficient approach to choose a smaller set of models to consider. There are three classical approaches for this task:

- *Forward selection.* We begin with the *null model*—a model that contains an intercept but no predictors. We then fit p simple linear regressions and add to the null model the variable that results in the lowest RSS. We then add to that model the variable that results in the lowest RSS for the new two-variable model. This approach is continued until some stopping rule is satisfied.
- *Backward selection.* We start with all variables in the model, and remove the variable with the largest p -value—that is, the variable that is the least statistically significant. The new $(p - 1)$ -variable model is fit, and the variable with the largest p -value is removed. This procedure continues until a stopping rule is reached. For instance, we may stop when all remaining variables have a p -value below some threshold.
- *Mixed selection.* This is a combination of forward and backward selection. We start with no variables in the model, and as with forward selection, we add the variable that provides the best fit. We continue to add variables one-by-one. Of course, as we noted with the **Advertising** example, the p -values for variables can become larger as new predictors are added to the model. Hence, if at any point the p -value for one of the variables in the model rises above a certain threshold, then we remove that variable from the model. We continue to perform these forward and backward steps until all variables in the model have a sufficiently low p -value, and all variables outside the model would have a large p -value if added to the model.

forward
selection
null model

backward
selection

mixed
selection

Backward selection cannot be used if $p > n$, while forward selection can always be used. Forward selection is a greedy approach, and might include variables early that later become redundant. Mixed selection can remedy this.

Three: Model Fit

Two of the most common numerical measures of model fit are the RSE and R^2 , the fraction of variance explained. These quantities are computed and interpreted in the same fashion as for simple linear regression.

Recall that in simple regression, R^2 is the square of the correlation of the response and the variable. In multiple linear regression, it turns out that it equals $\text{Cor}(Y, \hat{Y})^2$, the square of the correlation between the response and the fitted linear model; in fact one property of the fitted linear model is that it maximizes this correlation among all possible linear models.

An R^2 value close to 1 indicates that the model explains a large portion of the variance in the response variable. As an example, we saw in Table 3.6 that for the **Advertising** data, the model that uses all three advertising media to predict **sales** has an R^2 of 0.8972. On the other hand, the model that uses only **TV** and **radio** to predict **sales** has an R^2 value of 0.89719. In other words, there is a *small* increase in R^2 if we include newspaper advertising in the model that already contains TV and radio advertising, even though we saw earlier that the p -value for newspaper advertising in Table 3.4 is not significant. It turns out that R^2 will always increase when more variables are added to the model, even if those variables are only weakly associated with the response. This is due to the fact that adding another variable always results in a decrease in the residual sum of squares on the training data (though not necessarily the testing data). Thus, the R^2 statistic, which is also computed on the training data, must increase. The fact that adding newspaper advertising to the model containing only TV and radio advertising leads to just a tiny increase in R^2 provides additional evidence that **newspaper** can be dropped from the model. Essentially, **newspaper** provides no real improvement in the model fit to the training samples, and its inclusion will likely lead to poor results on independent test samples due to overfitting.

By contrast, the model containing only **TV** as a predictor had an R^2 of 0.61 (Table 3.2). Adding **radio** to the model leads to a substantial improvement in R^2 . This implies that a model that uses TV and radio expenditures to predict sales is substantially better than one that uses only TV advertising. We could further quantify this improvement by looking at the p -value for the **radio** coefficient in a model that contains only **TV** and **radio** as predictors.

The model that contains only **TV** and **radio** as predictors has an RSE of 1.681, and the model that also contains **newspaper** as a predictor has an RSE of 1.686 (Table 3.6). In contrast, the model that contains only **TV** has an RSE of 3.26 (Table 3.2). This corroborates our previous conclusion that a model that uses TV and radio expenditures to predict sales is much more accurate (on the training data) than one that only uses TV spending. Furthermore, given that TV and radio expenditures are used as predictors, there is no point in also using newspaper spending as a predictor in the model. The observant reader may wonder how RSE can increase when **newspaper** is added to the model given that RSS must decrease. In general RSE is defined as

$$\text{RSE} = \sqrt{\frac{1}{n - p - 1} \text{RSS}}, \quad (3.25)$$

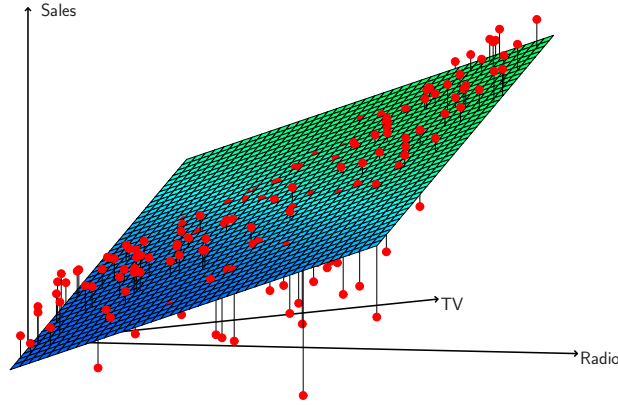


FIGURE 3.5. For the *Advertising* data, a linear regression fit to *sales* using *TV* and *radio* as predictors. From the pattern of the residuals, we can see that there is a pronounced non-linear relationship in the data. The positive residuals (those visible above the surface), tend to lie along the 45-degree line, where *TV* and *Radio* budgets are split evenly. The negative residuals (most not visible), tend to lie away from this line, where budgets are more lopsided.

which simplifies to (3.15) for a simple linear regression. Thus, models with more variables can have higher RSE if the decrease in RSS is small relative to the increase in p .

In addition to looking at the RSE and R^2 statistics just discussed, it can be useful to plot the data. Graphical summaries can reveal problems with a model that are not visible from numerical statistics. For example, Figure 3.5 displays a three-dimensional plot of *TV* and *radio* versus *sales*. We see that some observations lie above and some observations lie below the least squares regression plane. In particular, the linear model seems to overestimate *sales* for instances in which most of the advertising money was spent exclusively on either *TV* or *radio*. It underestimates *sales* for instances where the budget was split between the two media. This pronounced non-linear pattern suggests a *synergy* or *interaction* effect between the advertising media, whereby combining the media together results in a bigger boost to sales than using any single medium. In Section 3.3.2, we will discuss extending the linear model to accommodate such synergistic effects through the use of interaction terms.

Four: Predictions

Once we have fit the multiple regression model, it is straightforward to apply (3.21) in order to predict the response Y on the basis of a set of values for the predictors $X_1, X_2, \dots, X_p$. However, there are three sorts of uncertainty associated with this prediction.

1. The coefficient estimates $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ are estimates for $\beta_0, \beta_1, \dots, \beta_p$. That is, the *least squares plane*

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p$$

is only an estimate for the *true population regression plane*

$$f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p.$$

The inaccuracy in the coefficient estimates is related to the *reducible error* from Chapter 2. We can compute a *confidence interval* in order to determine how close $\hat{Y}$ will be to $f(X)$.

2. Of course, in practice assuming a linear model for $f(X)$ is almost always an approximation of reality, so there is an additional source of potentially reducible error which we call *model bias*. So when we use a linear model, we are in fact estimating the best linear approximation to the true surface. However, here we will ignore this discrepancy, and operate as if the linear model were correct.
3. Even if we knew $f(X)$ —that is, even if we knew the true values for $\beta_0, \beta_1, \dots, \beta_p$ —the response value cannot be predicted perfectly because of the random error ϵ in the model (3.20). In Chapter 2, we referred to this as the *irreducible error*. How much will Y vary from $\hat{Y}$? We use *prediction intervals* to answer this question. Prediction intervals are always wider than confidence intervals, because they incorporate both the error in the estimate for $f(X)$ (the reducible error) and the uncertainty as to how much an individual point will differ from the population regression plane (the irreducible error).

We use a *confidence interval* to quantify the uncertainty surrounding the *average sales* over a large number of cities. For example, given that \$100,000 is spent on **TV** advertising and \$20,000 is spent on **radio** advertising in each city, the 95 % confidence interval is [10,985, 11,528]. We interpret this to mean that 95 % of intervals of this form will contain the true value of $f(X)$.⁹ On the other hand, a *prediction interval* can be used to quantify the uncertainty surrounding *sales* for a *particular* city. Given that \$100,000 is spent on **TV** advertising and \$20,000 is spent on **radio** advertising in that city the 95 % prediction interval is [7,930, 14,580]. We interpret this to mean that 95 % of intervals of this form will contain the true value of Y for this city. Note that both intervals are centered at 11,256, but that the prediction interval is substantially wider than the confidence interval, reflecting the increased uncertainty about *sales* for a given city in comparison to the average *sales* over many locations.

confidence
interval

prediction
interval

⁹In other words, if we collect a large number of data sets like the **Advertising** data set, and we construct a confidence interval for the average *sales* on the basis of each data set (given \$100,000 in **TV** and \$20,000 in **radio** advertising), then 95 % of these confidence intervals will contain the true value of average *sales*.

3.3 Other Considerations in the Regression Model

3.3.1 Qualitative Predictors

In our discussion so far, we have assumed that all variables in our linear regression model are *quantitative*. But in practice, this is not necessarily the case; often some predictors are *qualitative*.

For example, the **Credit** data set displayed in Figure 3.6 records variables for a number of credit card holders. The response is **balance** (average credit card debt for each individual) and there are several quantitative predictors: **age**, **cards** (number of credit cards), **education** (years of education), **income** (in thousands of dollars), **limit** (credit limit), and **rating** (credit rating). Each panel of Figure 3.6 is a scatterplot for a pair of variables whose identities are given by the corresponding row and column labels. For example, the scatterplot directly to the right of the word “Balance” depicts **balance** versus **age**, while the plot directly to the right of “Age” corresponds to **age** versus **cards**. In addition to these quantitative variables, we also have four qualitative variables: **own** (house ownership), **student** (student status), **status** (marital status), and **region** (East, West or South).

Predictors with Only Two Levels

Suppose that we wish to investigate differences in credit card balance between those who own a house and those who don’t, ignoring the other variables for the moment. If a qualitative predictor (also known as a *factor*) only has two *levels*, or possible values, then incorporating it into a regression model is very simple. We simply create an indicator or *dummy variable* that takes on two possible numerical values.¹⁰ For example, based on the **own** variable, we can create a new variable that takes the form

factor
level
dummy
variable

$$x_i = \begin{cases} 1 & \text{if } i\text{th person owns a house} \\ 0 & \text{if } i\text{th person does not own a house,} \end{cases} \quad (3.26)$$

and use this variable as a predictor in the regression equation. This results in the model

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{if } i\text{th person owns a house} \\ \beta_0 + \epsilon_i & \text{if } i\text{th person does not.} \end{cases} \quad (3.27)$$

Now β_0 can be interpreted as the average credit card balance among those who do not own, $\beta_0 + \beta_1$ as the average credit card balance among those who do own their house, and β_1 as the average difference in credit card balance between owners and non-owners.

Table 3.7 displays the coefficient estimates and other information associated with the model (3.27). The average credit card debt for non-owners is estimated to be \$509.80, whereas owners are estimated to carry \$19.73 in additional debt for a total of $\$509.80 + \$19.73 = \$529.53$. However, we

¹⁰In the machine learning community, the creation of dummy variables to handle qualitative predictors is known as “one-hot encoding”.

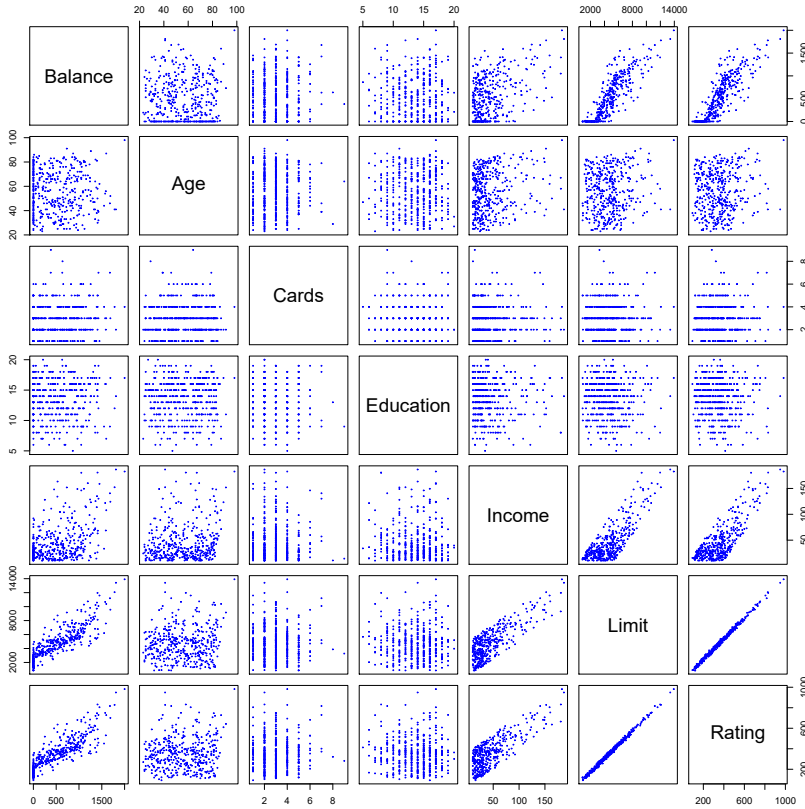


FIGURE 3.6. The **Credit** data set contains information about **balance**, **age**, **cards**, **education**, **income**, **limit**, and **rating** for a number of potential customers.

notice that the p -value for the dummy variable is very high. This indicates that there is no statistical evidence of a difference in average credit card balance based on house ownership.

The decision to code owners as 1 and non-owners as 0 in (3.27) is arbitrary, and has no effect on the regression fit, but does alter the interpretation of the coefficients. If we had coded non-owners as 1 and owners as 0, then the estimates for β_0 and β_1 would have been 529.53 and -19.73 , respectively, leading once again to a prediction of credit card debt of $\$529.53 - \$19.73 = \$509.80$ for non-owners and a prediction of $\$529.53$ for owners. Alternatively, instead of a 0/1 coding scheme, we could create a dummy variable

$$x_i = \begin{cases} 1 & \text{if } i\text{th person owns a house} \\ -1 & \text{if } i\text{th person does not own a house} \end{cases}$$

and use this variable in the regression equation. This results in the model

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{if } i\text{th person owns a house} \\ \beta_0 - \beta_1 + \epsilon_i & \text{if } i\text{th person does not own a house.} \end{cases}$$

	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	509.80	33.13	15.389	< 0.0001
own[Yes]	19.73	46.05	0.429	0.6690

TABLE 3.7. *Least squares coefficient estimates associated with the regression of balance onto own in the Credit data set. The linear model is given in (3.27). That is, ownership is encoded as a dummy variable, as in (3.26).*

Now β_0 can be interpreted as the overall average credit card balance (ignoring the house ownership effect), and β_1 is the amount by which house owners and non-owners have credit card balances that are above and below the average, respectively.¹¹ In this example, the estimate for β_0 is \$519.665, halfway between the non-owner and owner averages of \$509.80 and \$529.53. The estimate for β_1 is \$9.865, which is half of \$19.73, the average difference between owners and non-owners. It is important to note that the final predictions for the credit balances of owners and non-owners will be identical regardless of the coding scheme used. The only difference is in the way that the coefficients are interpreted.

Qualitative Predictors with More than Two Levels

When a qualitative predictor has more than two levels, a single dummy variable cannot represent all possible values. In this situation, we can create additional dummy variables. For example, for the **region** variable we create two dummy variables. The first could be

$$x_{i1} = \begin{cases} 1 & \text{if } i\text{th person is from the South} \\ 0 & \text{if } i\text{th person is not from the South,} \end{cases} \quad (3.28)$$

and the second could be

$$x_{i2} = \begin{cases} 1 & \text{if } i\text{th person is from the West} \\ 0 & \text{if } i\text{th person is not from the West.} \end{cases} \quad (3.29)$$

Then both of these variables can be used in the regression equation, in order to obtain the model

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{if } i\text{th person is from the South} \\ \beta_0 + \beta_2 + \epsilon_i & \text{if } i\text{th person is from the West} \\ \beta_0 + \epsilon_i & \text{if } i\text{th person is from the East.} \end{cases} \quad (3.30)$$

Now β_0 can be interpreted as the average credit card balance for individuals from the East, β_1 can be interpreted as the difference in the average balance between people from the South versus the East, and β_2 can be interpreted as the difference in the average balance between those from the West versus the East. There will always be one fewer dummy variable than the number of levels. The level with no dummy variable—East in this example—is known as the *baseline*.

baseline

¹¹Technically β_0 is half the sum of the average debt for house owners and the average debt for non-house owners. Hence, β_0 is exactly equal to the overall average only if the two groups have an equal number of members.

	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	531.00	46.32	11.464	< 0.0001
region[South]	-12.50	56.68	-0.221	0.8260
region[West]	-18.69	65.02	-0.287	0.7740

TABLE 3.8. Least squares coefficient estimates associated with the regression of **balance** onto **region** in the **Credit** data set. The linear model is given in (3.30). That is, **region** is encoded via two dummy variables (3.28) and (3.29).

From Table 3.8, we see that the estimated **balance** for the baseline, East, is \$531.00. It is estimated that those in the South will have \$18.69 less debt than those in the East, and that those in the West will have \$12.50 less debt than those in the East. However, the *p*-values associated with the coefficient estimates for the two dummy variables are very large, suggesting no statistical evidence of a real difference in average credit card balance between South and East or between West and East.¹² Once again, the level selected as the baseline category is arbitrary, and the final predictions for each group will be the same regardless of this choice. However, the coefficients and their *p*-values do depend on the choice of dummy variable coding. Rather than rely on the individual coefficients, we can use an *F*-test to test $H_0 : \beta_1 = \beta_2 = 0$; this does not depend on the coding. This *F*-test has a *p*-value of 0.96, indicating that we cannot reject the null hypothesis that there is no relationship between **balance** and **region**.

Using this dummy variable approach presents no difficulties when incorporating both quantitative and qualitative predictors. For example, to regress **balance** on both a quantitative variable such as **income** and a qualitative variable such as **student**, we must simply create a dummy variable for **student** and then fit a multiple regression model using **income** and the dummy variable as predictors for credit card balance.

There are many different ways of coding qualitative variables besides the dummy variable approach taken here. All of these approaches lead to equivalent model fits, but the coefficients are different and have different interpretations, and are designed to measure particular *contrasts*. This topic is beyond the scope of the book.

contrast

3.3.2 Extensions of the Linear Model

The standard linear regression model (3.19) provides interpretable results and works quite well on many real-world problems. However, it makes several highly restrictive assumptions that are often violated in practice. Two of the most important assumptions state that the relationship between the predictors and response are *additive* and *linear*. The additivity assumption means that the association between a predictor X_j and the response Y does not depend on the values of the other predictors. The linearity assumption states that the change in the response Y associated with a one-unit change in X_j is constant, regardless of the value of X_j . In later chapters of this book, we examine a number of sophisticated methods that relax these two

additive
linear

¹²There could still in theory be a difference between South and West, although the data here does not suggest any difference.

assumptions. Here, we briefly examine some common classical approaches for extending the linear model.

Removing the Additive Assumption

In our previous analysis of the **Advertising** data, we concluded that both **TV** and **radio** seem to be associated with **sales**. The linear models that formed the basis for this conclusion assumed that the effect on **sales** of increasing one advertising medium is independent of the amount spent on the other media. For example, the linear model (3.20) states that the average increase in **sales** associated with a one-unit increase in **TV** is always β_1 , regardless of the amount spent on **radio**.

However, this simple model may be incorrect. Suppose that spending money on radio advertising actually increases the effectiveness of TV advertising, so that the slope term for **TV** should increase as **radio** increases. In this situation, given a fixed budget of \$100,000, spending half on **radio** and half on **TV** may increase **sales** more than allocating the entire amount to either **TV** or to **radio**. In marketing, this is known as a *synergy* effect, and in statistics it is referred to as an *interaction* effect. Figure 3.5 suggests that such an effect may be present in the advertising data. Notice that when levels of either **TV** or **radio** are low, then the true **sales** are lower than predicted by the linear model. But when advertising is split between the two media, then the model tends to underestimate **sales**.

Consider the standard linear regression model with two variables,

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon.$$

According to this model, a one-unit increase in X_1 is associated with an average increase in Y of β_1 units. Notice that the presence of X_2 does not alter this statement—that is, regardless of the value of X_2 , a one-unit increase in X_1 is associated with a β_1 -unit increase in Y . One way of extending this model is to include a third predictor, called an *interaction term*, which is constructed by computing the product of X_1 and X_2 . This results in the model

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \epsilon. \quad (3.31)$$

How does inclusion of this interaction term relax the additive assumption? Notice that (3.31) can be rewritten as

$$\begin{aligned} Y &= \beta_0 + (\beta_1 + \beta_3 X_2) X_1 + \beta_2 X_2 + \epsilon \\ &= \beta_0 + \tilde{\beta}_1 X_1 + \beta_2 X_2 + \epsilon \end{aligned} \quad (3.32)$$

where $\tilde{\beta}_1 = \beta_1 + \beta_3 X_2$. Since $\tilde{\beta}_1$ is now a function of X_2 , the association between X_1 and Y is no longer constant: a change in the value of X_2 will change the association between X_1 and Y . A similar argument shows that a change in the value of X_1 changes the association between X_2 and Y .

For example, suppose that we are interested in studying the productivity of a factory. We wish to predict the number of **units** produced on the basis of the number of production **lines** and the total number of **workers**. It seems likely that the effect of increasing the number of production lines